

超低时延密码杂凑算法 LLH

设计与评估

冯秀涛、曾祥勇、王张宜、徐圣源、向泽军、王焕东、
唐明、唐灯、秦波、高博、张卫国、陈思维

2026.06.25

目 录

1	算法概述	6
2	算法设计原理	9
2.1	整体设计思想	9
2.2	置换 P 设计原理	11
2.3	主要组件设计原理	12
2.3.1	超级 S 盒 S_{24}	12
2.3.2	S 盒 S_6	12
2.3.3	错位排列与线性变换	13
2.3.4	行移位	13
2.3.5	常数加	14
2.3.6	小结	14
3	算法组件性质	14
3.1	整体结构性质	14
3.2	基础组件性质	15
3.2.1	S 盒 S_6	15
3.2.2	错位排列	16
3.2.3	线性变换	16
3.2.4	行移位	17
3.2.5	轮常数序列	17
3.3	超级 S 盒	17
3.4	轮函数	18
3.5	全扩散性质	19

3.6	代数次数性质	19
4	算法安全性分析	23
4.1	整体结构安全性分析	23
4.1.1	安全模型与假设	23
4.1.2	不可区分性	24
4.1.3	抗碰撞性证明	26
4.2	消息长度扩展分析	29
4.2.1	基本长度扩展分析	29
4.2.2	碰撞长度扩展分析	30
4.3	抗原像恢复分析	30
4.3.1	基于 SAT 的原像恢复分析	31
4.3.2	基于方程组的线性化求解分析	35
4.4	抗第二原像恢复分析	36
4.5	抗碰撞分析	45
4.6	侧信道分析	46
4.6.1	侧信道分析对象	46
4.6.2	平台正确性验证	47
4.6.3	针对 24 比特超级 S 盒的模板攻击可行性分析	49
4.7	抗量子计算分析	51
5	算法随机性统计测试	52
5.1	测试对象与测试设置	53
5.2	基础统计结果	54
5.3	NIST SP 800-22 统计测试结果	54
5.4	雪崩效应测试结果	56

5.5	综合结论	57
6	软硬件性能评估	58
6.1	软件性能评估	58
6.1.1	Intel/Amd 平台	58
6.1.2	Arm 平台	72
6.2	硬件性能评估	86
6.2.1	硬件实现	86
6.2.2	运算效率	87
6.2.3	硬件实现面积(12nm)	87
6.2.4	硬件时延(12nm)	88
6.2.5	吞吐量	88
6.2.6	归一化吞面比	89
6.2.7	功耗	89
6.2.8	性能分析	90
7	算法创新性和优势	91
7.1	算法创新性	91
7.2	算法优势	92
8	结论	93
	致谢	95
	参考文献	96

1 算法概述

LLH 是一族基于海绵结构[1]变体设计的密码杂凑算法，支持长度不超过 2^{64} 比特的输入消息，并提供 256、512、768 和 1024 比特四种标准输出长度。算法核心组件是一个定义在 $24 \times w$ 比特状态上的置换函数 $\mathcal{P}$ ，其采用切片技术设计，通过多轮迭代实现非线性混淆与全状态扩散。LLH 算法的整体结构如图 1-1 所示。

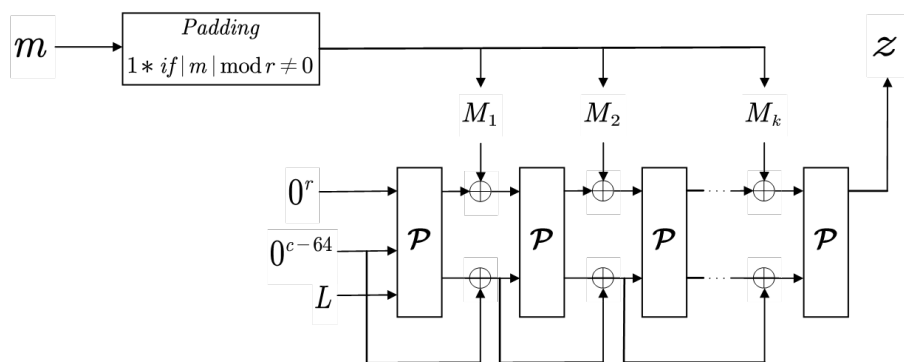


图 1-1 LLH 算法整体结构图

LLH 算法提供四种不同杂凑值长度的实例，见表 1-1。设字宽为 w ，则算法内部状态比特宽度为 $b=24w$ ，比特率 $r=8w$ ，容量 $c=16w$ ，杂凑值长度 $d=8w$ 。

表 1-1 LLH 标准实例参数配置

实例名称 (Variant)	字宽 w	内部状态 大小 b	典型输出 长度 d	比特率 (Rate)	容量 (Capacity)	轮数 N_r
LLH-256	32	768	256	256	512	16
LLH-512	64	1536	512	512	1024	24
LLH-768	96	2304	768	768	1536	32
LLH-1024	128	3072	1024	1024	2048	40

设输入消息为 M ，其长度为 L 比特。算法首先对消息进行填充，使其长度成为 r 的整数倍。若 $L \bmod r=0$ ，则不进行填充；否则在消息末尾附

加若干个比特 1，直至总长度达到不小于 L 的最小 r 的整数倍。设填充后的消息记为

$$M' = M_1 \parallel M_2 \parallel \cdots \parallel M_k,$$

其中每个消息块 M_i 的长度均为 r 比特。随后进行初始化，初始状态定义为

$$S^{(0)} = 0^{b-64} \parallel L,$$

其次，算法进入吸收阶段。对第 i 个消息块 M_i ，先对当前状态 $S^{(i-1)}$ 执行置换 P ，记 $T=P(S^{(i-1)})$ ，再将该消息块通过异或方式注入状态 T 的前 r 比特，即比特率部分，而状态 T 的后 c 比特与 $S^{(i-1)}$ 的后 c 比特异或，即

$$S^{(i)}=P(S^{(i-1)})\oplus(M_i \parallel S_{r:b-1}^{(i-1)}), \quad 1 \leq i \leq k.$$

最后，当所有消息吸收完毕，算法进入挤压阶段，此时算法执行置换

$$S^{(k+1)}=P(S^{(k)}),$$

最终输出杂凑值为

$$Z = S_{0:r-1}^{(k+1)}.$$

置换函数 P 是 LLH 的核心运算，其输入与输出均为一个 $24 \times w$ 比特状态。设 N_r 表示轮数，则 P 由 N_r 个相同结构的轮函数迭代构成。记输入状态为 $X^{(0)}$ ，第 t 轮后的状态为 $X^{(t+1)}$ ，则有

$$X^{(t+1)} = RoundFun(X^{(t)}), \quad 0 \leq t < N_r,$$

其中，轮函数由列替换、行移位以及常数加三个操作组成：

- **列替换**。将状态分成 w 个切片 C_j ，每个切片进行一个 24 比特超级 S 盒 $\mathcal{S}_{24}$ 变换。设超级 S 盒的输入为 $(v_0, v_1, \cdots, v_{23})$ ， $\mathcal{S}_{24}$ 定义如下：
 - 1) 首先，将 24 比特输入编码为一个 6×4 的比特矩阵：

$$M_{6 \times 4} = \begin{pmatrix} v_0 & v_1 & v_2 & v_3 \\ v_4 & v_5 & v_6 & v_7 \\ v_8 & v_9 & v_{10} & v_{11} \\ v_{12} & v_{13} & v_{14} & v_{15} \\ v_{16} & v_{17} & v_{18} & v_{19} \\ v_{20} & v_{21} & v_{22} & v_{23} \end{pmatrix}.$$

2) 其次，对该矩阵的每一列分别作用一个 6 比特 S 盒 $\mathcal{S}_6$ ：

$$\mathcal{S}_6(x) = x \oplus (x \ggg 2) \oplus ((x \ggg 1) \wedge (x \ggg 3)) \oplus 0x1B.$$

3) 再次，对所得 24 比特进行错位排列：

表 1-2 错位排列

0	1	2	3	错位排列 →	4	9	18	23
4	5	6	7		8	1	22	15
8	9	10	11		12	17	6	3
12	13	14	15		20	5	10	19
16	17	18	19		0	21	14	11
20	21	22	23		16	13	2	7

4) 最后，将错位排列后的矩阵每一行视作一个 4 比特行向量

R_i ($0 \leq i < 6$)，右乘一个 4×4 的矩阵 D ：

$$D = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}.$$

- **行移位。**将状态的每一行 W_i 看成是一个 w 比特的字，对字进行循环左移运算。第 i 个字关于 w 的移位偏移量由表 1-3 给出，于是

$$W_i \leftarrow W_i \lll \sigma_i.$$

表 1-3 行循环移位 σ_i 具体取值

i	$w = 32$	$w = 64$	$w = 96$	$w = 128$
0	3	2	2	1
1	16	25	11	16

2	13	20	28	55
3	26	51	53	118
4	8	15	19	10
5	29	40	34	29
6	2	37	57	72
7	23	6	88	11
8	21	36	52	35
9	18	63	73	58
10	31	62	6	105
11	28	33	43	48
12	10	1	5	76
13	15	30	32	103
14	4	31	67	26
15	9	4	14	101
16	7	38	70	5
17	20	5	7	36
18	17	8	48	91
19	30	47	1	42
20	12	19	55	78
21	1	52	94	113
22	6	57	45	44
23	27	34	4	127

- 常数加。对状态的最后一个字 W_{23} 异或一个常数：

$$W_{23} \leftarrow W_{23} \oplus RC_t,$$

其中 RC_t 为第 t 轮的 w 比特轮常数，其为自然底数 e 的小数部分二进制表示截取的第 tw 到 $(t+1)w$ 之间的比特。

2 算法设计原理

2.1 整体设计思想

LLH 算法采用海绵结构变体设计，具有结构简洁、软硬件实现友好以及高安全、低时延等特点。LLH 算法内部状态宽度 $b = 24w$ ，比特率为 $r = 8w$ ，容量为 $c = 16w$ ，输出长度为 $d = 8w$ 。在该参数下，

$$\frac{c}{2} = 8w = d,$$

根据海绵结构相关理论，该参数设置可以提供理想化的结构安全边界。

LLH 算法整体设计主要遵循以下原则：

- **结构简洁性原则。**LLH 算法整体采用海绵结构变体，在保持原海绵结构优点的同时进一步提高了抵抗与消息长度有关的攻击的能力。置换 P 采用切片技术设计，由列替换、行移位和常数加三个操作构成，其中，列替换采用经典 SPN 结构构造。这种布局使得整体结构简洁紧凑，便于设计。
- **超低时延设计原则。**LLH 算法在整个设计过程中始终遵循超低时延设计原则。这种原则不仅体现在算法结构选型方面，还包括所有密码组件的选择。首先，在核心结构上我们选择了比特切片技术构造置换 P 。比特切片易于并行实现，是同时具有高安全和低时延特性的主流密码结构。其次，我们精心构建了在保证密码学性质前提下时延达到数学理论下界的密码组件。例如：基础非线性部件 $\mathcal{S}_6$ ，其分支数为 3，电路实现深度为 2，达到非线性置换函数电路实现深度的数学理论下界；0-1 线性变换 D ，其分支数达到理论上界 4，同时电路实现深度也达到数学理论下界 2。由此构成的 24 比特超级 S 盒 $\mathcal{S}_{24}$ 的分支数为 5，电路实现深度为 4，是当前已知的大尺寸 S 盒中电路实现深度最低的。此外，错位排列和行移位均不引入额外电路逻辑深度，因此轮函数的电路实现深度为 4，它也是当前已知的电路实现深度最低的轮函数。
- **软硬件实现友好原则。**LLH 算法的所有操作均面向字，全部由循环移位、按位与、异或等低成本且易于实现的基础运算组成，因此 LLH 算法非常便于软硬件实现。
- **并行原则。**LLH 算法在结构安排上充分考虑了并行计算需求，尤其适合基于 SIMD 的并行实现。置换 P 采用切片技术构造，对字

进行逻辑运算可以自然地实现所有切片的并行处理。其次，字与字之间以及字之内均采用相似的操作，例如异或、与或者循环移位，这些操作方便通过采用 SIMD 的向量指令一次对多个字进行同时处理，有效提高了 LLH 算法的软件优化实现效率。

- **透明性原则。**LLH 在设计中尽量避免使用缺乏明确来源或难以解释的任意性参数，强调结构和参数选择的公开性、可验证性与可复现性。算法的主要组件均具有明确的构造依据，轮常数的生成规则也保持透明，从而避免隐藏参数或经验性调参对整体设计可信度造成影响。

2.2 置换 P 设计原理

置换函数 P 是 LLH 算法的核心部件，其输入状态被组织为一个 $24 \times w$ 的比特矩阵，其中第 i 行对应一个 w 比特字 W_i ，第 j 列对应一个 24 比特切片。基于切片技术，置换函数 P 重复执行列替换、行移位和常数加三个步骤。这种结构设计的主要考虑在于：

- 结构简洁，各操作目标明确，便于设计。其中，列替换主要实现非线性混淆和列内的局部扩散，行移位实现列之间的扩散，常数加则破坏不同轮函数之间可能存在的对称性和周期性传播模式。
- 有利于实现低时延设计目标。由于行移位和常数加在硬件实现逻辑上时延可以忽略不计，因此实现低时延的核心任务集中到构造低时延的列替换即可，这种处理方式有利于低时延设计目标的实现。针对列替换，为了降低大尺寸 S 盒的构造难度，同时便于软硬件实现和实现低时延设计目标，我们进一步选择了 SPN 结构，其中 S 盒层和线性层拟选择密码性质良好的时延达到数学理论下界的密码组件，

从而保持较高安全性的同时达到超低时延的设计目标。

- 便于软硬件实现和 SIMD 指令并行实现。整个置换函数 P 仅采用面向字的异或、与和循环移位等低成本且易于实现的基础运算，软硬件实现友好，代价低，且有利于 SIMD 指令并行实现。

2.3 主要组件设计原理

2.3.1 超级 S 盒 $\mathcal{S}_{24}$

超级 S 盒 $\mathcal{S}_{24}$ 主要实现对切片非线性置换，其设计遵循以下原则：

- 非线性置换
- 具有极低的电路实现深度
- 具有好的混淆和扩散效果
- 便于软硬件实现，且硬件成本低

基于上述原则，我们采用 SPN 结构构造超级 S 盒 $\mathcal{S}_{24}$ ：首先，将 24 比特输入编码成一个 6 行 4 列的比特矩阵；其次，对该矩阵依次作列 S 盒变换、错位排列和行扩散三种变换；最后，对变换后的矩阵重新编码得到 $\mathcal{S}_{24}$ 的输出。

上述分层设计一方面便于将低时延约束逐层落实到基础组件，另一方面可以低成本地实现好的混淆和扩散效果，便于密码性质的可分析性。

2.3.2 S 盒 $\mathcal{S}_6$

6 比特 S 盒 $\mathcal{S}_6$ 是置换 P 中最基础、也是最关键的非线性部件。其设计的主要原则如下：

- 非线性置换
- 没有不动点
- 电路实现深度达到数学理论下界 2

- 较好的扩散效果、差分均匀度和非线性度
- 实现简单，成本低

为构造满足上述要求的非线性 S 盒，我们研究了循环电路的一般形式，给出了一类具有扩散分支数为 3、电路实现为 2 的大尺寸非线性置换的构造方法，参考定理 2-1[14]。该构造比原始循环电路具有更好的扩散效果。

定理 2-1： 设 $n=3k$ ， k 为大于 1 的整数。则

$$f(x) = x \oplus (x \ggg 2) \oplus (x \ggg 1) \cdot (x \ggg 3)$$

是 $\mathbb{Z}_2^n$ 上的置换。

在我们的具体构造中选择 $k=2$ ， $n=6$ 。

2.3.3 错位排列与线性变换

错位排列与线性变换 D 共同构成列替换中的扩散部分。错位排列的作用是在不增加逻辑深度的前提下，尽可能打散输入矩阵中的行、列相关性，因此要求全部位置均发生错位，并使原同一行与同一列中的元素在排列后尽量分布到不同的行中，同时使排列周期达到最大。对于线性变换 D ，其设计主要考虑是在极小成本下实现尽可能强的扩散能力。相应地， D 被构造成一个分支数达到理论最佳、电路实现深度也达到理论最佳的 4 比特线性变换。二者结合后，使列替换能够在保持低时延的同时获得较强的局部扩散效果。

2.3.4 行移位

行移位的主要作用是将列替换内部形成的局部活跃模式扩展到整个状态。各行字的移位量互不相同，由简单的二元多项式

$$\sigma(x, y) = a_0 + a_1x + a_2y + a_3x^2 + a_4xy + a_5y^2 \pmod{w}$$

生成，且周期达到最大。对单个切片对应的 6×4 比特状态矩阵，要求同一列内至多 2 个字的移位量模 4 同余，同一行内各字移位量模 4 两两不同，确保排列结构无简单等差规律。该设计以尽可能低的成本完成跨列扩散，使局部超级 S 盒中形成的非线性在多轮迭代后迅速覆盖全部状态。

2.3.5 常数加

常数加的设计目标则是以最小代价破坏轮间对称性。轮常数仅注入到状态的最后一个字，其生成方式透明、无周期，并具有较好的表观随机性。这种设计避免了在全状态范围内引入复杂常数层，同时仍能有效抑制对称传播和短周期轨道对整体状态演化的不利影响。

2.3.6 小结

总体而言，LLH 的各个组件并非分别追求单一指标最优，而是在低时延、低成本、软硬件实现友好和密码学性质良好等约束下进行协同设计。基础组件尽量保持简单，通过组合获得更强的局部混淆与扩散，并通过整体轮结构迭代再将这种局部传播提升为全状态扩散。正是在这一设计思路下，LLH 形成了其面向超低时延实现的整体架构。

3 算法组件性质

3.1 整体结构性性质

从结构层面看，LLH 采用海绵结构变体设计，内部状态宽度为

$$b = 24w,$$

其中比特率、容量和输出长度分别为

$$r=8w, \quad c=16w, \quad d=8w.$$

因此，容量所对应的理想化结构安全边界为

$$\frac{c}{2}=8w=d.$$

这表明，在当前参数配置下，结构层面的安全边界与目标输出长度相匹配。默认情况下，算法在完成消息吸收后执行一次挤压即可得到最终摘要；更一般地，挤压过程也可以参数化为 t 次，但总输出长度保持为 $d=8w$ 。这一结构安排既保留了海绵结构变体清晰的安全边界，又为不同接口形式下的实现保留了参数化空间。

此外，消息长度 L 作为初始输入块提前进入 P 置换，这使得传统的与消息长度有关的攻击方法均失效，例如长度扩展攻击等，进一步提高了算法抗攻击的安全能力。

3.2 基础组件性质

3.2.1 S 盒 $\mathcal{S}_6$

6 比特 S 盒 $\mathcal{S}_6$ 由函数

$$\mathcal{S}_6(x)=x\oplus(x\ggg 2)\oplus((x\ggg 1)\wedge(x\ggg 3))\oplus 0x1B.$$

给出，其是循环电路函数的一种推广。设输入 $x=x_0x_1x_2x_3x_4x_5$ ，输出为 $y=y_0y_1y_2y_3y_4y_5$ ，则该函数的电路实现如图 3-1 所示。

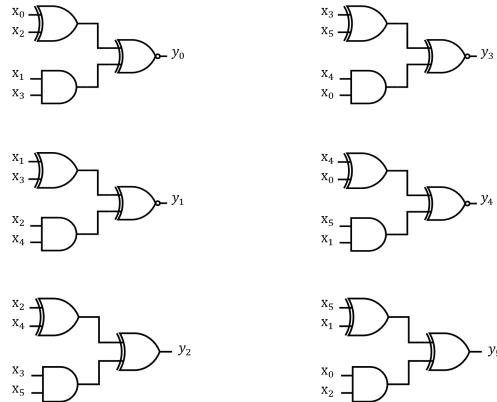


图 3-1 S 盒函数电路实现图

S 盒 $\mathcal{S}_6$ 具有如下性质：

- 逻辑深度为 2，达到非线性密码置换函数深度的数学理论下界；
- 分支数为 3，并在深度达到最优的同时实现该扩散指标的最优；
- 差分均匀度为 16；
- 非线性度为 16；
- 无不动点。

同 SHA-3 的 5 比特循环电路函数 χ 的密码学性质对比见表 3-1。

表 3-1 S_6 与 SHA-3 循环电路函数 χ 的性质对比

项目	S_6	χ
分支数	3	2
差分均匀度	16	8
非线性度	16	8

3.2.2 错位排列

错位排列通过将输入矩阵中的元素位置打散使得差分可以传播到其它整个切片中。其具有如下的性质：

- 错位排列；
- 每列元素进行错位重排，且周期达到最大 6；
- 任意一行元素重排后分布到不同的行中。

3.2.3 线性变换

线性变换 D 是 0-1 矩阵，其电路实现如图 3-2 所示。

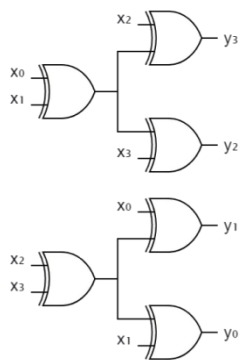


图 3-2 线性变换 D 的电路实现图

D 满足如下性质：

- 分支数为 4，达到 4 阶 0-1 矩阵分支数的最优；
- 电路实现深度为 2，同样达到理论最佳；
- 实现上仅需 6 个异或门。

3.2.4 行移位

行移位具有如下性质：

- 行移位 $w_i = \sigma(x, y)$ 是环 Z_w 上关于二元变量 (x, y) 的置换多项式，其中 $x = i \gg 2$ ， $y = i \bmod 4$ ， i 是位置索引；
- 不同 w 对应的置换周期达到最大 w ；
- 像之间不存在简单的等差性质。

3.2.5 轮常数序列

轮常数序列通过自然底数 e 的小数部分的二进制表示序列截取得到，具有透明、无显著周期和表观随机性较好的特点。常数仅注入最后一个字，代价极低，但足以破坏轮间对称性。

3.3 超级 S 盒

超级 S 盒 S_{24} 是一个 24 比特到 24 比特的置换，它具有如下性质：

- 分支数为 5；

- 电路实现深度为 4；
- 每个输出比特依赖于 12 个输入比特；
- 不存在概率非零的仅第一行输入差分与输出差分同时非零的差分对。

超级 S 盒关于输入输出差分重量的分布见表 3-2。由该分布可见，低重量输入输出差分对的分布受到明显限制，而较高重量传播占据更主要位置。

表 3-2 S_{24} 的差分重量分布

1	0	0	0	0	0	100663296	0	0	201326592	0	0	100663296	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	2156824	13280182	0	25794906	184548376	138413832	478150656	303494916	506870912	496733348	533488128	643825684	360701104	88080384	176160768	90177536	0	0	0	0	0	
3	0	0	18087360	87031808	108003336	567779616	1077411840	1449132032	1087181456	4506409440	6625730024	4508698784	4423945144	3691618253	1598852157	969408	1182703725	322961408	0	0	0	0	0	
4	0	0	4860736	14837304	31082842	85668632	1811132864	422369280	930667296	206855847	446830694	445968086	913397580	506569271	5360849	10530808	10440020	1048576	0	0	0	0	0	
5	0	0	6912000	18333508	34545472	258066656	37940311	5596180992	21834844	1099408588	7046546452	70367300	131377361	181909736	52220886	1015064576	170265696	278109286	3622633472	657457152	154337280	24739840	196608	
6	98384	10148736	34181092	195706034	732646632	600030	144874044	277297484	485701428	9265378076	1167574576	64023887	121487065	1545745817	55661998	2070001402	28780200	456028456	88638320	14147008	6386368	0	0	
7	2064384	286818304	83426688	146173824	70850140	493451100	2031049726	35283248	470704526	65065020895	350583639	52072707	1691583459	26081056839	1085185656	938024781	1083764531	889029002	190698827	206363664	547381248	45347520	0	
8	1592548	320251264	443463168	915648512	1914330734	3031528448	6122374348	7889545216	12362549648	21342084	203673664	308239027	441305494	633870451	647711359	4480861	154291177	1123206972	1411889557	84320217	5776896	6211872	0	
9	5266488	34142064	263623664	540320072	446154467	633077454	1647310484	5468066	154911795	32621257	474231677	12026001	121206721	17123396974	4141888532480	3227557260	1461506091	102905100	354842624	103172096	7501824	1382400	0	
10	9012544	34357304	3639816192	97986110	972236406	1059490817	292066676	19583447	748273967	45892800280	20280251599	180642376	692969091	54874686	402741494	4942042	4597236868	479234688291	2346889291	2346889291	544832	4490352	0	
11	42840726	444631168	915648512	1914330734	3031528448	6122374348	7889545216	12362549648	21342084	203673664	308239027	441305494	633870451	647711359	4480861	154291177	1123206972	1411889557	84320217	5776896	6211872	0	0	
12	7687378	335800812	540320072	446154467	633077454	1647310484	5468066	154911795	32621257	474231677	12026001	121206721	1712339697	4141888532	322755726	1461506091	102905100	354842624	103172096	7501824	1382400	0	0	
13	4069756	596651076	478620076	594236256	647918571	53217924	502674855	5045374	430625573	626318379	2347814378	55886462	4535947	45456183	18536176	3533222548035	401794461	1060949028	10977344	84953207	36088216	6996898	0	
14	2854208	44171532	298408632	487704664	679431772	1763253336	254916436	614306631	101480970387	15184928440	4523151	1033331401	578713170	8613623259	391361	196814044	5920027	40484902028	368639212	252331814	123393734	36088216	0	
15	2654208	44171532	298408632	487704664	679431772	1763253336	254916436	614306631	101480970387	15184928440	4523151	1033331401	578713170	8613623259	391361	196814044	5920027	40484902028	368639212	252331814	123393734	36088216	0	
16	0	0	2906994	1808087880	4201608831	324502601	643478796	4548076	443262525	723206321	1164022604	959738701	641710327	2550188	18996576	160694696	10965480	1662404	969696301	101808081	1622020	7788009	277816	1793436
17	0	0	15178888	89371680	4201570204	5923169878	439321620	710097051	762882504	9444907	1265934480	46581406	164043481	65542829627	770425006	48293795	100549969	33358184	13175991	13175991	34224	103172096	7501824	0
18	0	0	4487020	3094560	4662400	1462400	697501696	1486211072	2667229184	5931357184	65799654987	152805016	541104064	680682967	12125694918	1848733584	718113713	125694918	1013638280	42834	2623084	0	0	
19	0	0	2101812	107835932	511793200	10218801	1396068	730688471	2142030131	521464567	1477530031	104841	41228813060	14718178796	605633312	1923038	354566070	107372288	38197720	74881024	14249984	98204	74881024	0
20	0	0	5749824	22569856	48745392	10218801	1396068	730688471	2142030131	521464567	1477530031	104841	41228813060	14718178796	605633312	1923038	354566070	107372288	38197720	74881024	14249984	98204	74881024	0
21	0	0	2101812	107835932	511793200	10218801	1396068	730688471	2142030131	521464567	1477530031	104841	41228813060	14718178796	605633312	1923038	354566070	107372288	38197720	74881024	14249984	98204	74881024	0
22	0	0	5749824	22569856	48745392	10218801	1396068	730688471	2142030131	521464567	1477530031	104841	41228813060	14718178796	605633312	1923038	354566070	107372288	38197720	74881024	14249984	98204	74881024	0
23	0	0	2101812	107835932	511793200	10218801	1396068	730688471	2142030131	521464567	1477530031	104841	41228813060	14718178796	605633312	1923038	354566070	107372288	38197720	74881024	14249984	98204	74881024	0
24	0	0	5749824	22569856	48745392	10218801	1396068	730688471	2142030131	521464567	1477530031	104841	41228813060	14718178796	605633312	1923038	354566070	107372288	38197720	74881024	14249984	98204	74881024	0
25	0	0	138240	223584	342016	1256768	2676576	91028328	20732356	602824320	62082456	720658784	69836448	542767194	36202496	294763136	9284982	3801296	11255808	2863104	514048	31764	0	0
26	0	0	36864	270336	134536	2691072	8146944	18130044	31125504	47861760	59387904	64499712	60182048	46608384	32206848	11448960	7999488	3502080	884736	307200	30720	0	0	0
27	0	0	0	0	30720	0	23124	0	1832160	0	3889152	0	583216	0	3962880	0	1451520	0	272384	0	21504	0	0	0
28	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
29	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
30	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
31	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
32	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
33	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
34	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
35	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
36	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
37	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
38	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
39	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
40	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
41	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
42	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
43	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
44	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
45	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
46	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
47	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
48	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
49	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
50	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
51	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
52	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
53	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
54	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
55	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
56	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
57	0	0	0	0	0																			

3.4 轮函数

单轮轮函数的性质如下：

- 分支数为 5；
 - 电路实现深度为 4；
 - 每个输出比特依赖于 12 个输入比特。
- 对两轮迭代而言，扩散能力显著增强：
- 分支数提高到 13；
 - 每个输出比特依赖于 144 个输入比特。

这一结果表明，经过两轮组合后，局部列内扩散已经在行移位作用下迅速传播到更大范围，使不同列之间形成明显耦合。换言之，LLH 的扩散并不依赖大量轮数缓慢积累，而是在较少轮次内即可实现显著增长。

3.5 全扩散性质

从全状态传播角度看, LLH 在不同参数实例下均能在较少轮数内达到全扩散。各实例对应的全扩散轮数见下表。

表 3-3 LLH 算法各版本全扩散轮数

LLH-256	LLH-512	LLH-768	LLH-1024
4	4	5	5

这一结果说明, 尽管字长 w 的增大会带来更大的状态规模, 但得益于列替换与行移位之间的协同作用, 活跃模式仍能在较少轮次内覆盖全状态。对于以超低时延为目标的杂凑算法而言, 这一点尤为关键: 它表明 LLH 并未因追求较浅逻辑层次而显著削弱整体扩散效率。相反, 算法能够在 4 至 5 轮范围内完成全状态扩散, 从而为安全冗余轮数的选取提供了较好的基础。

总体来看, LLH 的组件性质呈现出清晰的层次结构: $\mathcal{S}_6$ 提供最低时延的基础非线性, D 提供最优的小规模线性扩散; $\mathcal{S}_{24}$ 将局部非线性 $\mathcal{S}_6$ 与局部扩散 D 整合为一个具有较强传播能力的超级 S 盒; 轮函数中列替换与行移位共同实现活跃模式在不同切片之间快速扩散。正是借助不同层次上的组件相互配合, 共同构成了 LLH 算法的安全性与超低时延的理论基础。

3.6 代数次数性质

LLH 算法作为海绵结构变体算法, 算法采用了多个置换, 其中核心置换采用 SPN 型结构设计, 且使用代数次数仅为 2 的 S 盒。本节将基于可分性评估算法 4 个置换正向和逆向的代数次数增长规律, 结果见表 3-4。当前评估 SPN 型结构代数次数的方法包括: 基于公式的理论评估

法、基于可分性的评估法等。文献[2]的结果表明：基于可分性的代数次数评估方法是所有评估方法中最为精确的评估法。因此，本节 LLH 算法的代数次数评估将采用可分性的评估法

令 $f, g: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ 表示两个向量布尔函数， $h = g \circ f$ 表示两个函数的复合函数。进一步，假设 g 的代数次数为 $\text{Deg}(g) = d$ ，且 M 表示 f 的可分性传播模型，其目标函数是当固定输入可分性时输出可分性的最小值。通过将输入可分性初始固定为 d 并依次增加输入可分性的值，记当输出可分性的最小值首次大于 d 时的输入可分性的值为 in ，则复合函数 h 的代数次数的评估值为 $in - 1$ 。上述方法需要对 LLH 算法公开置换进行可分性建模并求解，然而 LLH 算法针对不同版本的置换宽度分别为 768、1536、2304、3072，上述宽度限制了基于比特可分性评估方法的效率，因此后续将采用基于字的可分性传播评估 LLH 算法的代数次数。

为了表述方便，令 E_i 表示 LLH 算法第 i 轮置换。首先固定输入可分性的汉明重量为 $in=1$ ，构建 $h=I \circ E_1$ (I 表示恒等映射) 基于字的可分性传播模型 M_{E_1} 。 M_{E_1} 约束输入可分性的汉明重量为 in ，并约束输出可分性的汉明重量小于等于 $out=1$ 。如果模型 M_{E_1} 有解，则将 in 加 1 后再次求解。若模型无解，则 E_1 的代数次数为 $in-1$ 。当面向两轮置换时，构建 $h=E_2 \circ E_1$ ，并将上述过程重复即可求得两轮公开置换的代数次数，具体步骤见**算法 3-1**。

算法描述：

第 3-4 行：设置输入可分性的汉明重量 in 与输出可分性汉明重量的下界 out 。

第 6 行：构建模型 M_{in}^{out} ， M_{in}^{out} 为公开置换一轮的基于字的可分性传播模型，其中输入可分性的汉明重量为 in ，输出可分性汉明重量小于等

于 out 。

第 7-13 行：求解模型 M_{in}^{out} ，如果模型有解，则增加输入可分性的汉明重量 in ，如果模型无解，则退出循环并输出轮数与代数次数。

算法 3-1 LLH 的基于字的可分性代数次数评估算法

```
1  :   $in=2$ 
2  :  for  $round \leftarrow 1$  to  $r$  do
3  :       $out \leftarrow in-1$ 
4  :       $in \leftarrow in-1$ 
5  :      while True do
6  :          Construct the model  $M_{in}^{out}$ 
7  :          optimize  $M_{in}^{out}$ 
8  :          if model is feasible
9  :               $in \leftarrow in+1$ 
10:          else
11:              break
12:          end if
13:      print( $round, in-1$ )
16:  end for
```

根据评估的代数次数，对于宽度为 768 比特的置换，可以构造 $14+9=23$ 轮的零和区分器，复杂度 2^{767} ；对于宽度为 1536 比特的置换，可以构造 $15+10=25$ 轮的零和区分器，复杂度 2^{1534} ；对于宽度为 2304 比特的置换，可以构造 $16+11=27$ 轮的零和区分器，复杂度 2^{2303} ；对于宽度为 3072 比特的置换，可以构造 $17+12=29$ 轮的零和区分器，复杂度 2^{3071} 。

表 3-4 LLH 置换迭代代数次数

方向		正向				逆向			
字长参数		w=32	w=64	w=96	w=128	w=32	w=64	w=96	w=128
轮数	r=1	2	2	2	2	3	3	3	3
	r=2	4	4	4	4	9	9	9	9
	r=3	8	8	8	8	27	27	27	27
	r=4	16	16	16	16	81	81	81	81
	r=5	32	32	32	32	243	243	243	243
	r=6	64	64	64	64	614	729	729	729
	r=7	128	128	128	128	729	1334	1842	1970
	r=8	256	256	256	256	758	1485	2188	2796
	r=9	512	512	512	512	765	1523	2275	3003
	r=10	682	1024	1024	1024	767	1532	2296	3054
	r=11	739	1365	1792	2048	767	1535	2302	3067
	r=12	758	1479	2133	2730	767	1535	2303	3070
	r=13	764	1517	2247	2958	767	1535	2303	3071
	r=14	766	1529	2285	3034	767	1535	2303	3071
	r=15	767	1533	2297	3059	767	1535	2303	3071
	r=16	767	1535	2301	3067	767	1535	2303	3071
	r=17	767	1535	2303	3070	767	1535	2303	3071
	r=18	767	1535	2303	3071	767	1535	2303	3071

此外，LLH 算法采用的海绵结构变体吸收消息时，消息字异或到了状态 24 个字的前 8 个字，因此在实际使用中只有状态的前 8 个字的值可以被任意修改。考虑将状态的前 8 个字作为变量评估输出关于状态前 8 个字的代数次数时， P 置换的 S 盒的代数次数为 2，并且 2 次项通过将一个比特与其间隔的比特相乘获得。因此，将状态的前 8 个字作为变量时，一轮输出的代数次数为 1。此时，表 3-4 中评估的代数次数结果同样使用，只需将对应的轮数+1 即可。

4 算法安全性分析

4.1 整体结构安全性分析

本节对 LLH 算法整体结构给出形式化安全性证明。LLH 算法与标准海绵结构的区别主要在于：

1. 初始状态不再为全 0，而是在容量区嵌入消息长度；
2. 对于置换函数 P 的迭代，加入前馈机制。

在理想置换模型下，本节证明 LLH 算法整体结构依然满足：

1. 伪随机不可区分性（PRF）；
2. 抗碰撞性。

4.1.1 安全模型与假设

LLH 算法整体结构的安全性证明基于理想置换模型（Ideal Permutation Model, IPM）[3]，该模型是海绵结构安全性证明的标准框架，能有效隔离底层置换的实现细节，聚焦于结构本身的安全性。同时，为确保证明的有效性，假设底层置换函数 P 是一个“完美”的随机置换，满足以下三个关键特性：

- （1）**双射**：对于任意输入 $X \in \{0,1\}^n$ ，存在唯一的输出 $Y = P(X) \in \{0,1\}^n$ ，且不同输入对应不同输出；反之，其逆置换 P^{-1} 也满足 $P^{-1}(P(X)) = X$ 。
- （2）**均匀随机性**：对于任意未查询过的输入 X ， $P(X)$ 的输出是均匀分布在 $\{0,1\}^n$ 上的随机串，与历史查询的输入、输出均无关；逆置换 P^{-1} 的输出同样满足均匀随机性。
- （3）**谕言机可访问性**：敌手可通过谕言机自由查询 P 和 P^{-1} ，每次查询后获得对应的输出，但无法获取置换的内部运算逻辑，也无法预

测未查询输入的输出。

4.1.2 不可区分性

命题 1: LLH 为伪随机函数 (Pseudorandom Function, PRF), 即不存在多项式时间敌手 $\mathcal{A}$, 能以不可忽略的优势区分 LLH 输出与真随机函数输出。

证明: 定义两个实验游戏, 通过敌手 $\mathcal{A}$ 的区分优势衡量 LLH 的伪随机性:

- (1) 真实游戏 (Game_0): 敌手 $\mathcal{A}$ 输入任意消息 m , 谕言机 $\mathcal{O}_0$ (LLH 算法) 按照 LLH 整体结构的“初始化-吸收-挤压”流程, 计算并返回杂凑值 Z ;
- (2) 理想游戏 (Game_1): 敌手 $\mathcal{A}$ 输入任意消息 m , 谕言机 $\mathcal{O}_1$ (真随机函数), 随机生成并输出一个与 Z 长度相同的二进制串 Z' 。注意, $\mathcal{O}_1$ 对于同一消息 m 多次查询, 返回的 Z 保持一致 (对于随机函数的定义, 可假设 $\mathcal{O}_1$ 具有记忆)。

区分优势定义为

$$\text{Adv}_{\mathcal{A}}^{\text{ind}} = |\Pr[\mathcal{A}^{\mathcal{O}_0}(1^\lambda)=1] - \Pr[\mathcal{A}^{\mathcal{O}_1}(1^\lambda)=1]|,$$

其中 λ 为安全参数, $\mathcal{A}^{\mathcal{O}_0}$ 表示敌手 $\mathcal{A}$ 访问 LLH 谕言机 $\mathcal{H}$, $\mathcal{A}^{\mathcal{O}_1}$ 表示敌手 $\mathcal{A}$ 访问真随机函数谕言机 $\mathcal{R}$ 。若 $\text{Adv}_{\mathcal{A}}^{\text{ind}}$ 为可忽略函数, 即随着安全参数 λ 的增大, 优势趋近于 0, 则 LLH 满足伪随机不可区分性。

下面采用归约证明方法: 假设存在多项式时间敌手 $\mathcal{A}$, 能以不可忽略的优势 ϵ 区分 Game_0 与 Game_1 , 则可构造一个归约算法 RED, 利用 $\mathcal{A}$ 的区分能力, 攻破底层置换 P 的理想性 (即区分 P 与真随机置换)。这与“ P 为理想置换”的假设矛盾, 从而证明不存在这样的敌手 $\mathcal{A}$ 。

步骤 1：归约算法RED的谕言机模拟

归约算法RED接收一个未知的谕言机 $\mathcal{O}$ （该谕言机要么是理想置换 P ，要么是真随机置换 $\mathcal{R}_P$ ）。RED的目标是由敌手 $\mathcal{A}$ ，判断 $\mathcal{O}$ 是 P 还是 $\mathcal{R}_P$ 。

对于理想游戏（Game₁），模拟 LLH 的谕言机 $\mathcal{O}_0$ ，接收敌手 $\mathcal{A}$ 的消息查询 M ，执行以下操作：

- （1）对消息 M 执行填充操作 $M' = \text{Padding}(M) = M_1, M_2, \dots, M_k$ ，得到分组序列 $M_1, M_2, \dots, M_k$ ；
- （2）计算原始消息长度 $L = \text{len}(M)$ ，这里 L 编码为64比特的二进制序列。
初始化初始状态 $S^{(0)} = 0^r || 0^{c-64} || L$ ；
- （3）依次处理每个分组 M_i ($i = 1, 2, \dots, k$):
 - 计算当前轮次的置换输入 $S^{(i)} = P(S^{(i-1)}) \oplus (M_i || S_{r:b-1}^{(i-1)})$ ；
 - RED 在每个轮次中，仅传递 P 的输入输出。
 - RED 全程将 P 作为黑盒调用，自身不执行 P 的内部运算。
- （4）截取最终状态 $S^{(k)}$ 的前 r 比特，作为杂凑值输出 z ，返回给敌手 $\mathcal{A}$ ；
- （5）敌手 $\mathcal{A}$ 输出一个判断结果（1 表示认为访问的是 LLH 谕言机 $\mathcal{O}_0$ ，0 表示认为访问的是真随机函数谕言机 $\mathcal{O}_1$ ，RED直接将该结果作为自身的输出，用于判断未知谕言机 $\mathcal{O}$ 的类型）。

步骤 2：区分优势的归约

分两种情况分析归约算法RED的优势：

情况 1：未知谕言机 $\mathcal{O} = P$ （理想置换）。此时， $\mathcal{R}$ 模拟的谕言机与Game₀中的 $\mathcal{O}_0$ 完全一致，敌手 $\mathcal{A}$ 输出 1 的概率为 $\Pr[\mathcal{A}^{\mathcal{O}_0}(1^\lambda)=1]$ 。

情况 2：未知谕言机 $\mathcal{O} = \mathcal{R}_P$ （真随机置换）。此时， $\mathcal{R}$ 模拟的谕言

机中，每个状态 $S^{(i)}$ 均为均匀随机串，且与已查询过的输入输出独立。由于填充规则满足唯一可解码性，不同消息的分组序列不同，对应的 $S^{(i)}$ 也不同，因此每个 $S^{(i)}$ 都是独立均匀随机的。最终输出 $Z' = S_{0:r-1}^{(k+1)}$ 也是均匀随机串，与 Game_1 中的 $\mathcal{O}_1$ 输出完全一致，敌手 $\mathcal{A}$ 输出 1 的概率为 $\Pr[\mathcal{A}^{\mathcal{O}_1}(1^\lambda)=1]$ 。

因此，归约算法RED对于置换函数 P 和随机置换 $\mathcal{R}_{\mathcal{P}}$ 的区分优势为：

$$Adv_{\mathcal{R}}^{perm} = |\Pr[\mathcal{R}^P(1^\lambda) = 1] - \Pr[\mathcal{R}^{\mathcal{R}_{\mathcal{P}}}(1^\lambda) = 1]| = Adv_{\mathcal{A}}^{ind} = \epsilon.$$

但根据假设， P 是理想置换，不存在多项式时间算法能以不可忽略的优势区分 P 与 $\mathcal{R}$ ，即 $Adv_{\mathcal{R}}^{perm}$ 必须是可忽略函数。这与“ $\mathcal{A}$ 以不可忽略优势 ϵ 区分两个游戏”的假设矛盾，因此不存在这样的敌手 $\mathcal{A}$ 。

□

关于嵌入消息长度的讨论。这里需要强调的是，初始状态中嵌入的消息长度 $L = \text{len}(M)$ 不会影响不可区分性。因为消息长度 L 是敌手已知的信息（敌手输入消息 M 时，必然知道其长度）， $L = \text{len}(M)$ 仅用于区分不同长度的消息，不会向敌手泄露任何额外信息，也不会导致状态的可区分性——无论是否嵌入 $L=\text{len}(M)$ ，在理想置换模型下，状态的随机性均由 P 是理想置换保证。

4.1.3 抗碰撞性证明

命题 2： LLH 整体结构的抗碰撞安全强度为 $\min(d/2, c/2)$ 比特。

碰撞场景分类

敌手寻找碰撞的核心是构造两个不同的消息 $M \neq M'$ ，使得它们经过“填充-吸收-挤压”流程后，最终输出的杂凑值相同。根据消息长度和

分组序列的差异，将碰撞场景分为两类，分别讨论每类场景下碰撞概率可忽略。

设消息 M 和 M' 经填充后，分别得到分组序列 $M_1, M_2, \dots, M_k$ 和 $M_1', M_2', \dots, M_k'$ ，对应的初始状态分别为 $S^{(0)}$ 和 $S'^{(0)}$ ，吸收阶段的状态序列分别为 $S^{(0)}, S^{(1)}, \dots, S^{(k)}$ 和 $S'^{(0)}, S'^{(1)}, \dots, S'^{(k)}$ ，最终输出分别为 $Z = S_{0:r-1}^{(k+1)}$ 和 $Z' = S'_{0:r-1}^{(k+1)}$ 。

场景 1：消息长度不同 ($\text{len}(M) \neq \text{len}(M')$)

若 M 和 M' 的长度不同 ($\text{len}(M) \neq \text{len}(M')$)，则它们的长度编码 $L \neq L'$ 。根据初始状态的定义：

$$S^{(0)} = 0^r || 0^{c-64} || L.$$

由于 $\text{len}(M) \neq \text{len}(M')$ ，因此 $S^{(0)} \neq S'^{(0)}$ 。

接下来分析吸收阶段的第一次状态更新：

$$S^{(1)} = P(S^{(0)}) \oplus (M_1 || S_{r:b-1}^{(0)}).$$

假设 $S^{(1)} = S'^{(1)}$ 。由于 P 是置换，满足 “ $P(X) = P(Y)$ 当且仅当 $X = Y$ ”。因此由 $S^{(0)} \neq S'^{(0)}$ 可知 $P(S^{(0)}) \neq P(S'^{(0)})$ ，而 M_1 和 M_1' 由攻击者根据 $P(S^{(0)})$ 和 $P(S'^{(0)})$ 的差值，选择为：

$$M_1 \oplus M_1' = P(S^{(0)})_{0:r-1} \oplus P(S'^{(0)})_{0:r-1}.$$

由此可将 r 区的差分消去。因此攻击者需要对于 $P(S^{(0)})$ 构造从 r 到 $b-1$ 比特（即 c 区）的局部碰撞，基于生日攻击的穷举攻击复杂度为 $2^{c/2}$ 。因此对于随机置换 P ，抵抗局部碰撞的安全强度为 $c/2$ 比特。

另外，由于 $S^{(0)} = 0^r || 0^{c-64} || L$ ，所以

$$P(S^{(0)})_{b-64:b-1} \oplus L = P(S^{(1)})_{b-64:b-1}.$$

如果攻击者通过尝试改变消息长度来改变 L 的值，进而消去 c 区中最后 64 比特的差分，将导致随机置换 P 的输入发生改变，进而使得的输出改变。随机置换 P 仅需要满足不存在最后 64 比特的不动点即可。

类似的，后续每一轮的状态更新中，基于生日攻击的穷举攻击复杂度为 $2^{c/2}$ 。对于随机置换 P ，抵抗局部碰撞的安全强度为 $c/2$ 比特。因此基于随机置换 P ，LLH 整体结构的抗碰撞安全强度为 $\min(d/2, c/2)$ 比特。

场景 2：消息长度相同但分组不同 ($\text{len}(M) = \text{len}(M')$, $k = k'$, 且存在 i 使得 $M_i \neq M'_i$)

若 M 和 M' 的长度相同 ($\text{len}(M) = \text{len}(M')$)，则它们的长度编码 $L = L'$ ，因此初始状态 $S^{(0)} = S'^{(0)}$ 。由于填充规则满足唯一可解码性， $M_i \neq M'_i$ 必然导致填充后的分组序列不同，即存在最小的索引 i ($1 \leq i \leq k$)，使得 $M_i \neq M'_i$ ，且对于所有 $j < i$ ，有 $M_j = M'_j$ 。

首先，根据吸收阶段的状态更新规则，已知：

(1) 当 $j = 0$ 时， $S^{(0)} = S'^{(0)}$ ；

(2) 对于 $j < i$ ， $M_j = M'_j$

可通过归纳法证明：对于 $j < i$ ， $S^{(j)} = S'^{(j)}$ 。

因此，当 $j = i - 1$ 时，有 $S^{(i-1)} = S'^{(i-1)}$ 。

接下来分析第 i 轮的状态更新：

$$S^{(i)} = P(S^{(i-1)}) \oplus (M_i \parallel S_{r:b-1}^{(i-1)}).$$

根据置换 P 的单值性，当 $X \neq Y$ 时有 $P(X) \neq P(Y)$ ，因此

$$P(S^{(i-1)}) \neq P(S'^{(i-1)}).$$

后续轮次中，由于 $S^{(i)} \neq S'^{(i)}$ ，且每一轮的状态更新都依赖于前一轮的状态，因此对于所有 $j > i$ ，均有 $S^{(j)} \neq S'^{(j)}$ 。最终状态 $S^{(k)} \neq S'^{(k)}$ ，输

出 $Z \neq Z'$ 。此类场景下，敌手为了消除差分构造碰撞，攻击者需要对于 $P(S^{(i)})$ 构造从 r 到 $b-1$ 比特（即 c 区）的局部碰撞，基于生日攻击的穷举攻击复杂度为 $2^{c/2}$ 。因此对于随机置换 P ，抵抗局部碰撞的安全强度为 $c/2$ 比特。随机置换 P 仅需要满足不存在最后 c 比特的不动点即可。

综合上述两种场景，敌手找到碰撞的总概率上界为

$$\max(2^{-d/2}, 2^{-c/2}),$$

其中， d 为最终杂凑值的比特长度。容量为 c 比特，用于保障算法的安全性，不直接与输入消息交互。

因此，LLH 整体结构的抗碰撞安全强度为 $\min(d/2, c/2)$ 比特。

□

具体对于 LLH-256、LLH-512、LLH-768、LLH-1024 四个版本的参数配置都有 $c=2d$ ，为容量区预留了充分的安全冗余。

4.2 消息长度扩展分析

4.2.1 基本长度扩展分析

长度扩展攻击针对 Merkle-Damgård 结构及标准海绵结构的核心缺陷[4]：敌手已知消息 M 的哈希值 z ，可利用标准海绵全 0 初始状态的特性，从吸收阶段的最终状态继续迭代，在原摘要值后面直接附加一段消息 M^* ，构造成类似 $M||M^*$ 的有效哈希值，无需知晓 M 的具体内容。

LLH 整体结构通过初始状态嵌入消息长度 L ，从根本上阻断长度扩展攻击的实施路径，原因如下：

- (1) 长度依赖性：扩展消息 $M||M^*$ 的长度为 $L' = L + |M^*|$ ，其初始状态为 $S^{(0)} = 0^r || 0^{c-64} || L'$ ，与原消息 M 的初始状态 $S^{(0)} = 0^r || 0^{c-64} || L$ 取值不同；

- (2) 状态不可逆性：底层置换 P 为单向置换，且增加了前馈机制，敌手无法从最终状态 $S^{(k)}$ 反推初始状态 $S^{(0)}$ ，更无法构造与 L' 匹配的初始状态；
- (3) 迭代独立性：原消息的状态迭代序列 $S^{(0)}, S^{(1)}, \dots, S^{(k)}$ 与扩展消息的迭代序列 $S'^{(0)}, S'^{(1)}, \dots, S'^{(k)}$ 不再相等，无法通过复用原迭代结果生成新杂凑值。

4.2.2 碰撞长度扩展分析

如果敌手已经找到一对碰撞 $M_1, M_2, \dots, M_k$ 和 $M_1', M_2', \dots, M_k'$ ，对于Merkle-Damgård 结构及标准海绵结构，后续可以基于该对碰撞，按照对应的迭代规则，分别附加相同内容的一段消息 M^* 到两组碰撞消息后面，从而构造出更多的碰撞对。

LLH 整体结构通过初始状态嵌入消息长度 L ，避免了这类攻击。如果敌手继续附加消息 M^* 到已经找到的碰撞消息后，将直接改变初始状态 $S^{(0)} = 0^r || 0^{c-64} || L$ 的取值。由新的初始状态 $S^{(0)}$ 重新计算 $M_1, M_2, \dots, M_k$ 和 $M_1', M_2', \dots, M_k'$ 杂凑值，对于随机置换 P ，仍是碰撞对的概率低于 $\max(2^{-d/2}, 2^{-c/2})$ 。

LLH 整体结构通过初始状态嵌入消息长度和前馈机制，相对于Merkle-Damgård 结构及标准海绵结构，改进了抵抗长度扩展攻击的安全性。

4.3 抗原像恢复分析

本节主要评估 LLH 约减轮版本是否存在优于枚举的原像恢复攻击方法。设 LLH 的输出长度为 $d = 8w$ ，则理想杂凑函数的枚举恢复原像的复杂度为 $2^d = 2^{8w}$ [5]。由于直接分析完整轮数的安全性较为困

难，下面首先考察低轮约减模型中的原像恢复方法及其复杂度，再据此评估此类攻击向完整版本延伸的可行性。

设单块消息 M_1 吸收后的状态为 S_0 ，之后依次记为

$$S_1 = f(S_0), \quad S_2 = f(S_1), \quad S_3 = f(S_2),$$

其中 f 表示一轮轮函数，如图 4-1 所示。单块消息 M_1 的 $8w$ 个比特以异或方式注入状态的前 8 个字，其余 16 个字保持为由初始化和反馈过程确定的常量。随着轮数增加，状态 S_i 关于消息变量的代数次数在低轮迭代过程中呈指数级增长，具体增长规律可参见 3.6 节；相应地，原像恢复问题意味着将由线性方程逐步转化为高次方程组求解问题。

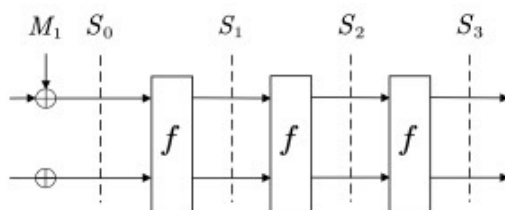


图 4-1 S_0 至 S_3 的状态演进示意图

下面分别从 CNF 建模 SAT 求解器求解和方程组线性化求解两个角度展开讨论。

4.3.1 基于 SAT 的原像恢复分析

SAT 分析的基本思路是：首先引入中间状态变量，然后建立中间状态变量之间以及其与输入消息、输出摘要之间简单约束关系，最后将它们之间的约束关系转化成 CNF 子句，并调用 SAT 求解器 Kissat 进行求解。

一轮情形。单块消息 M_1 以异或方式注入状态 S_0 的前 8 个字，如

图 4-2 所示。红色块表示受消息 M_1 影响的字，黑色块表示已知常量。由于每个 s_6 中的输入中只有 2 比特受消息影响，其余 4 比特为已知常量，根据 s_6 的定义，易知输出比特或者为常数，或者为输入消息的仿射函数，如图 4-2 右侧 24 个字的状态，其中黑色表示常量，白色为输入消息 M_1 的仿射函数，灰色表示条件常量，其或者为常量，或者为输入消息 M_1 的仿射函数，具体依赖于输入中常量部分的取值。对给定的输出摘要 z ，逆向经过错位排列、线性变换 D ，可以得到状态 T_0 的部分已知字，见图 4-3 黑色块所示。结合图 4-2 和图 4-3，我们可以得到状态 T_0 中关于第 1、4、18、22、23 等 5 个字的全部仿射方程以及第 15 个字的部分仿射方程。由于消息 M_1 共有 $8w$ 个比特，在平均意义下，每个输出摘要可以确定 M_1 中的 $5.5w$ 个消息比特，这说明，约 $2.5w$ 个消息比特是自由的，即对应一轮迭代，大约有 $2^{2.5w}$ 个消息映射到同一个摘要 z ，它们可以通过高斯消元求解线性方程组直接给出。

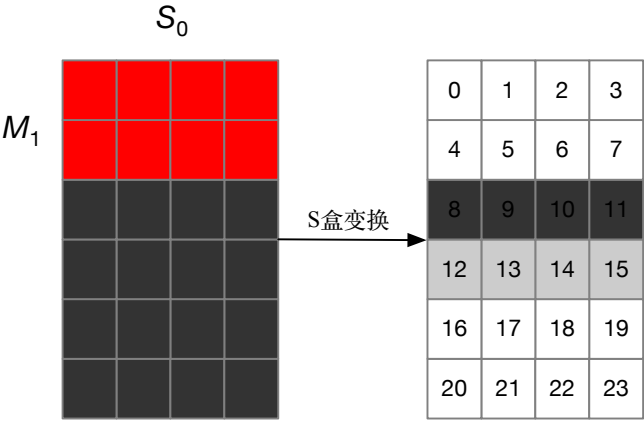


图 4-2 超级 S 盒状态转换图

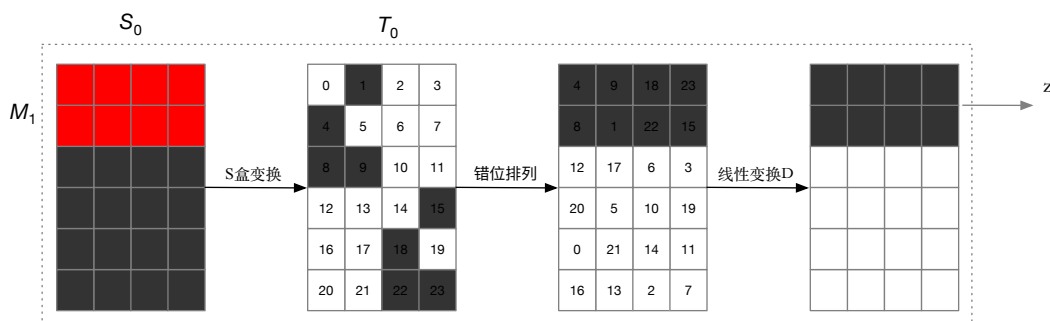


图 4-3 一轮轮函数迭代

两轮情形。首先引入中间状态变量 T_0 和 T_1 。和一类轮函数迭代类似，对给定的输出摘要 z ，逆向经过错位排列、线性变换 D ，可以得到状态 T_1 的部分已知字，见图 4-4 黑色块所示。根据轮函数定义，我们建立 T_0 与 M_1 ， S_1 与 T_0 ， T_1 黑色块与 S_1 之间的简单代数方程，其中， T_0 与 M_1 以及 S_1 与 T_0 之间的方程是线性的， T_1 黑色块与 S_1 之间的方程是二次的，并将它们转化为 CNF 子句，所得 SAT 实例包含约 2560 个变量和约 15300 个子句。最后在服务器 DELL T640 (2 CPU, 56 核, 512G 内存) 调用 Kissat 对其求解，在限定时间 12 小时内，Kissat 未返回 SAT 和 UNSAT。

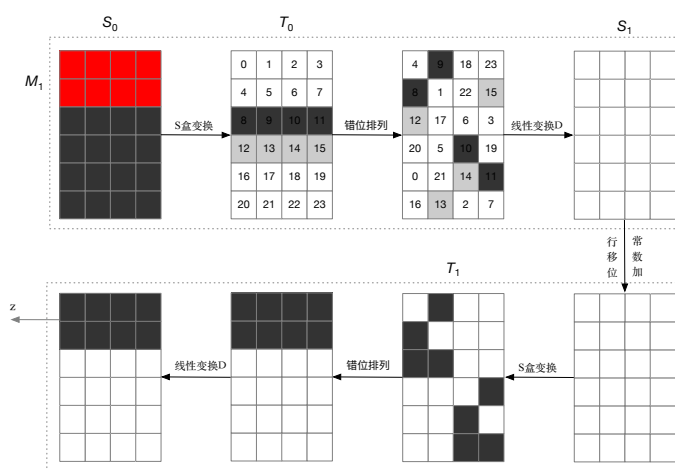


图 4-4 二轮轮函数迭代

三轮情形。和两轮迭代处理类似，我们首先引入中间状态变量 T_0 、 T_1 和 T_2 。同样对给定的输出摘要 z ，逆向经过错位排列、线性变换 D ，可以得到状态 T_2 的部分已知字，见图 4-5 黑色块所示。根据轮函数定义，我们建立 T_0 与 M_1 ， S_1 与 T_0 ， T_1 与 S_1 ， S_2 与 T_1 以及 T_2 黑色块与 S_2 之间的简单代数方程，其中， T_0 与 M_1 ， S_1 与 T_0 ， S_2 与 T_1 之间的方程是线性的， T_1 与 S_1 ， T_2 黑色块与 S_2 之间的方程是二次的，并将它们转化为 CNF 子句，所得 SAT 实例包含约 4860 个变量和约 30650 个子句。在限定时间 12 小时内，Kissat 未返回 SAT 和 UNSAT。

图 4-5 三轮轮函数迭代

理论上是不可行的。

4.3.2 基于方程组的线性化求解分析

本节从方程组结构出发，给出一种显式的猜测—线性化求解方法。一轮情形可以归结于线性方程组求解，参见上节，这里不再赘述。下面我们主要针对两轮情形展开讨论。

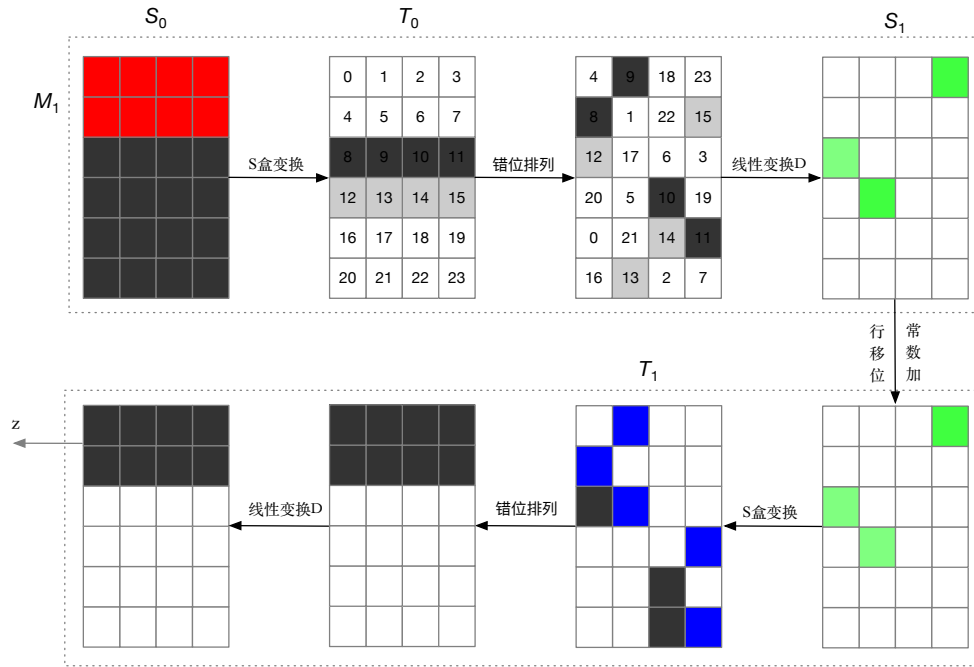


图 4-6 两轮置换结构示意图

首先我们猜测状态 S_1 的第 3, 8 和 13 位置的字，共 $3w$ 个比特，如图 4-6 所示绿色方格对应的字。由于状态 S_1 是输入消息 M_1 的仿射函数，因此我们根据猜测的值可以得到 $3w$ 个线性方程。其次，观察状态 T_1 中第一列的蓝色方块，其代数方程为：

$$y_1 = x_1 \oplus x_3 \oplus x_2 x_4 \oplus 1,$$

其中 x_2 恰好为状态 S_1 第 1 列第 3 个字，即第 1 列中的绿色方块，因其为猜测变量，于是上述方程退化为线性方程。再次，观察 T_1 中第二列的两

个蓝色方块，其代数方程为：

$$y_0 = x_0 \oplus x_2 \oplus x_1 x_3 \oplus 1,$$

$$y_2 = x_2 \oplus x_4 \oplus x_3 x_5 \oplus 1,$$

容易发现其二次项涉及公共变量 x_3 ，我们猜测 x_3 的值，即状态 S_1 第2列的绿色方块，上述两个方程同时退化为线性方程。类似地，我们猜测状态 S_1 第4列的绿色方块，同样可以使状态 T_1 第4列的两个蓝色方块退化为关于 S_1 的线性方程。于是，我们可以得到关于状态 T_1 中5个蓝色方块的线性方程。结合3个猜测字，实际上我们得到了 $8w$ 个线性方程。利用高斯消元求解上述线性方程组可以恢复消息 M_1 。上述分析的复杂度为 2^{3w} ，优于枚举的时间复杂度。

我们尝试将上述方法推广到三轮迭代，由于三轮迭代输出摘要 z 是关于输入消息 M_1 的4次函数，采用线性化方法需要猜测的比特变量的总数已经超过 $8w$ ，高于枚举攻击的复杂度，因此我们相信LLH能够很好的抵抗原像恢复攻击。

4.4 抗第二原像恢复分析

第二原像攻击的目标是在给定消息及其杂凑值的条件下，寻找另一条不同消息，使二者具有相同的输出杂凑值。由于LLH采用按块吸收的前馈海绵结构，若只考虑单块消息，则消息空间和输出杂凑值空间同为 d 比特。此时，约减模型中的置换 P 可近似看作随机置换，其诱导出的单块消息到杂凑值的映射大概率不存在非平凡第二原像。因此，低轮第二原像分析主要考虑两个消息块的情形。

设给定两块消息为 m_0, m_1 ，目标是寻找不同的 m'_0, m'_1 ，使最终杂凑

值保持不变。对一轮约减模型而言，经过一次列替换、行移位和常数加后，输出杂凑值可表示为输入消息变量的线性函数。因此，可以随机选取 $m'_0 \neq m_0$ ，随后将目标杂凑值保持不变作为线性方程组，反解出对应的 m'_1 。只要该线性系统可解，即可得到一个两块消息的第二原像。

下面进一步给出一条针对两块消息、两轮约减模型的差分型第二原像路径。分析以单个 24 比特切片为单位进行，集合表示该切片中的活跃行位置。例如， $\{0,2,4,7\}$ 表示第 0、2、4、7 行存在非零差分，对应 24 比特掩码为 $0x000095$ 。记

S6: 4 个并行 6-bit S 盒

shuffle: 固定错位排列

D: 4-bit 行线性变换

Super Sbox: $S6 \rightarrow \text{shuffle} \rightarrow D$

轮常数加法不影响异或差分，因此在差分概率分析中可以忽略。第二块消息吸收时，攻击者可以通过 Δm_1 控制相关切片的低 8 比特差分。

为便于理解后续差分路径，图中所有 6×4 网格均表示一个 24 比特切片，格子位置对应行编号，着色格表示该位置上存在非零差分。红色表示由第一块消息差分 Δm_0 产生的初始传播路径；绿色和青色分别表示行移位后形成的 Branch A 与 Branch B；白色表示不存在差分的位置。箭头上标注的概率仅来自 S 盒差分转移，shuffle、行移位和线性变换 D 均为确定性操作，不引入额外概率。

图 4-7 展示了两块消息、两轮约减模型中的完整差分结构。第一阶段，攻击者在 m_0 的某一列注入初始差分 $\Delta m_0 = \{0,2,4,7\}$ ，使其经过一个

超级 S 盒后以概率 2^{-8} 传播为{1,21}。第二阶段，行移位将{1}与{21}分散到两个不同列，形成 Branch A 和 Branch B。两条分支分别继续经过一个超级 S 盒，并在第二块消息吸收前形成固定的高 16 比特差分模式。虚线右侧第三阶段表示第二块消息 m_1 的吸收阶段，攻击者通过选择 Δm_1 的低 8 比特差分，使两条分支都进入相同的输出模式{14,20}。最后,{14,20}经行移位后转化为单比特截断差分，并在后续超级 S 盒与线性层作用下消除杂凑输出位置上的差分。

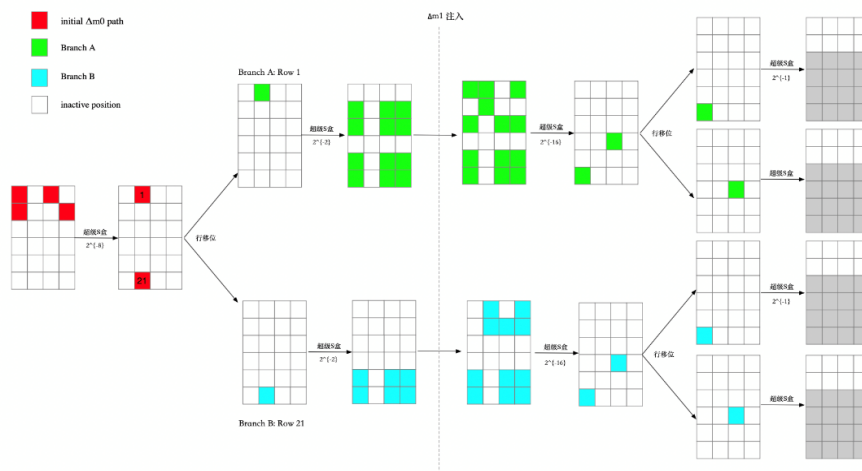


图 4-7 第二原像攻击总览

第一阶段，在 m_0 的某一列选择初始差分：

$$\Delta_{in} = 0x000095 \quad \{0,2,4,7\}$$

$$S6_{out} = 0x850094 \quad \{2,4,7,16,18,23\}$$

$$SuperSbox_{out} = 0x200002 \quad \{1,21\}$$

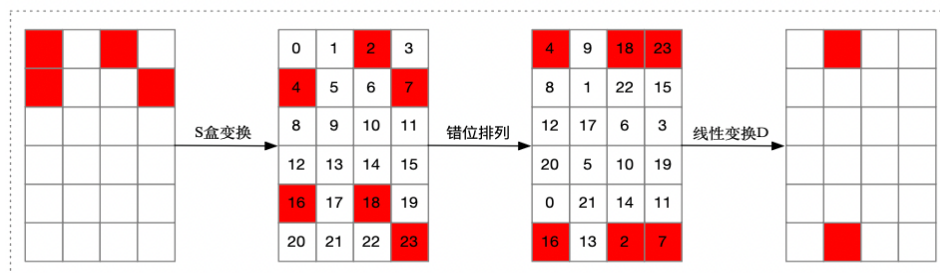


图 4-8 初始差分传播

其中，S6 层的 DDT 分解如下。

表 4-1 初始 S 盒 DDT 分解

列	输入差分	输出差分	DDT 数	概率
0	0x03	0x12	4	2^{-4}
1	0x00	0x00	64	1
2	0x01	0x11	16	2^{-2}
3	0x02	0x22	16	2^{-2}

图 4-8 给出了第一阶段的精确传播。输入差分{0,2,4,7}首先经过 4 个并行 6 比特 S 盒。图中第二个网格标出了 S 盒输出差分{2,4,7,16,18,23}；随后经过错位排列和线性变换 D，该差分被压缩为{1,21}。由于 shuffle 和 D 都是确定性操作，本阶段概率完全由 S 盒 DDT 决定，即 $2^{-4} \times 2^{-2} \times 2^{-2} = 2^{-8}$ 。

第一阶段输出{1,21}之后，行移位使 row 1 与 row 21 进入两个不同列。若两条分支所在列不发生碰撞，则后续传播概率可以近似相乘。对于 row 1 分支，选择如下路径：

Delta_in = 0x000002 {1}

S6_out = 0x222002 {1,13,17,21}

SuperSbox_out = 0xdd0dd0 {4,6,7,8,10,11,16,18,19,20,22,23}



图 4-9 分支 A 部分差分传播

S6 层中唯一非平凡列转移为 $0x01 \rightarrow 0x39$ ，DDT 计数为 16，对应概率 2^{-2} 。

对于 row 21 分支，选择如下路径：

$$\Delta_{in} = 0x200000 \quad \{21\}$$

$$S6_out = 0x202000 \quad \{13,21\}$$

$$SuperSbox_out = 0xdd0000 \quad \{16,18,19,20,22,23\}$$



图 4-10 分支 B 部分差分传播

S6 层中唯一非平凡列转移为 $0x20 \rightarrow 0x28$ ，DDT 计数为 16，对应概率 2^{-2} 。因此，两条分支在该阶段的合计概率为 2^{-4} 。

进入第二块消息吸收后，攻击者利用 Δm_1 调整两条相关列的低 8 比特差分，使完整输入差分落入有利路径。

对 row 1 分支，第一块传播在第二块吸收前留下的差分为 $0xdd0dd0$ 。在 m_1 对应列选择低 8 比特差分，使吸收后的完整输入差分为

Delta_in = 0xdd0d2b

随后选择精确传播：

S6_out = 0x1820a4 {2,5,7,13,19,20}

SuperSbox_out = 0x104000 {14,20}

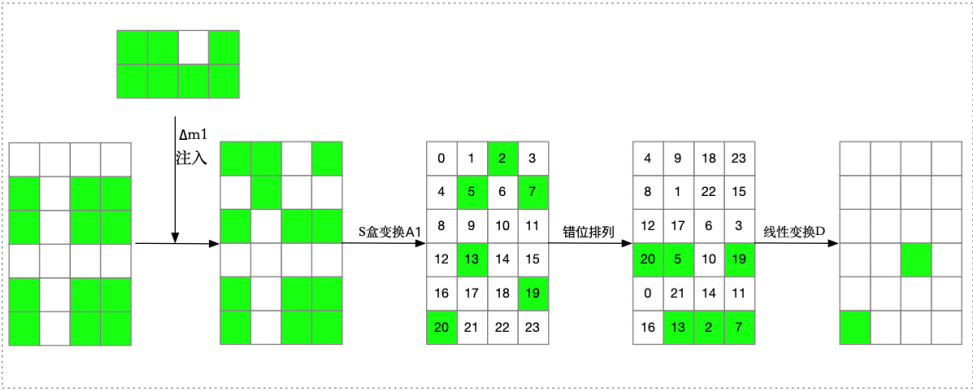


图 4-11 Δm_1 注入后分支 A 差分传播

其 S6 层 DDT 分解如下。

表 4-2 S 盒 A1 的 DDT 分解

列	输入差分	输出差分	DDT 数	概率
0	0x35	0x20	4	2^{-4}
1	0x03	0x0a	4	2^{-4}
2	0x34	0x01	4	2^{-4}
3	0x35	0x12	4	2^{-4}

因此该分支在第二块吸收后的传播概率为 2^{-16} 。

对 row 21 分支, 第一块传播在第二块吸收前留下的差分为 0xdd0000。

在 m_1 对应列选择低 8 比特差分 0xea = {1,3,5,6,7}, 使吸收后的完整输入差分为

Delta_in = 0xdd00ea

选择精确传播：

$S6_out = 0x1820a4 \quad \{2,5,7,13,19,20\}$

$SuperSbox_out = 0x104000 \quad \{14,20\}$

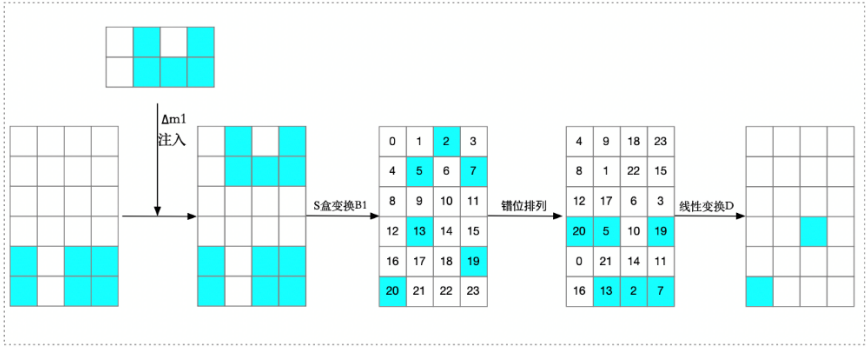


图 4-12 Δm_1 注入后分支 B 差分传播

其 S6 层 DDT 分解如下。

表 4-3 S 盒 B1 的 DDT 分解

列	输入差分	输出差分	DDT 数	概率
0	0x30	0x20	4	2^{-4}
1	0x03	0x0a	4	2^{-4}
2	0x32	0x01	4	2^{-4}
3	0x33	0x12	4	2^{-4}

因此该分支在第二块吸收后的传播概率同样为 2^{-16} 。

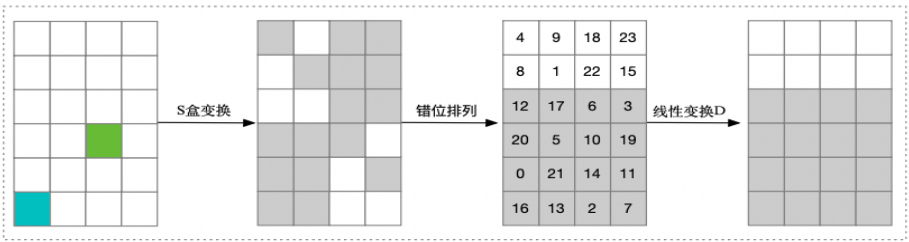


图 4-13 最终截断差分条件

图 4-13 展示了最终截断条件。经过 Δm_1 调整后，两条分支均输出

{14,20}。行移位将该差分拆分为单比特差分后，后续 S 盒输出在目标杂凑位置上满足截断为零的条件：{1,4,8,9,15,18,22,23} 上的差分为 0。分别计算这两个单比特差分经过下一层 S6 后满足截断条件的概率。其中，row 14 的单比特差分以概率 1 满足该截断条件；row 20 的单比特差分以概率 2^{-1} 满足该截断条件。因此，每条分支在最终截断阶段的概率为 2^{-1} ，两条分支合计为 2^{-2} 。最终，确定性的线性层将该截断条件传递到杂凑输出的前 8 比特，使这些输出位置上的差分消失。

将各阶段概率相乘，固定初始列时路径总成功概率为 $2^{-8} \times 2^{-2} \times 2^{-2} \times 2^{-16} \times 2^{-16} \times 2^{-2} = 2^{-46}$ 。

若攻击者可在 w 个列位置上尝试同类路径，并近似认为不同列尝试相互独立，则成功概率约为 $w \cdot 2^{-46}$ ，对应期望复杂度约为 $2^{46}/w$ 。对 LLH-256， $w = 32$ ，得到复杂度估计为 2^{41} 。

综上，在两块消息、两轮约减模型中，存在一条结构化第二原像差分路径：先通过 Δm_0 产生两个可分离的单比特分支，再通过 Δm_1 调整第二块吸收时的低 8 比特差分，使两条分支均压缩到 {14,20}，最后利用行移位后的单比特截断性质消除杂凑值输出差分。当路径命中时，攻击者可构造 $(m_0 \oplus \Delta m_0, m_1 \oplus \Delta m_1)$ 作为原两块消息 (m_0, m_1) 的第二原像候选。

高轮安全性讨论。上述路径说明，在两块消息、两轮约减模型中，攻击者可以利用第二块消息吸收阶段的低 8 比特自由度，对第一块传播产生的分支差分进行局部调整，从而构造一条结构化第二原像差分路径。然而，该路径强烈依赖于低轮模型中的弱扩散特性。随着轮数增加，差分会经过更多次列替换、行移位和线性变换，活跃位置会迅速扩散到

更多列和更多 S 盒中。此时，单个消息块中可控的低位差分很难再只影响少量局部分支，也难以同时补偿多个列中的高位固定差分。因此，两轮路径中的“先分裂、再用第二块低 8 比特局部修正”的机制在更高轮数下不再容易成立。

从扩散角度看，两轮模型中差分传播仍具有较强的局部性。攻击者能够跟踪 $\{1\}$ 和 $\{21\}$ 两条分支，并分别选择 Δm_1 使其进入目标模式 $\{14,20\}$ 。但在更多轮之后，行移位会不断把同一切片中的活跃位置分散到不同列，线性层又会在列内扩展活跃差分。结果是，差分路径需要同时满足越来越多 S 盒的转移约束，并且需要在更多列上满足最终截断条件。相比两轮情形，高轮模型中的自由度增长速度不足以抵消扩散带来的约束增长。

从最少活跃 S 盒数量也可以得到同样结论。设 A_R 表示 R 轮传播中任意非零差分路径至少需要激活的 S 盒数量。由于每个活跃 6 比特 S 盒的最大差分转移概率不超过 2^{-2} ，任意一条精确差分路径的概率均可上界为 $Pr[\text{path}] \leq 2^{-2 \cdot A_R}$ 。若还需要满足最终杂凑输出位置的截断条件，则实际成功概率只会更低。因此，攻击复杂度至少随 2^{2A_R} 增长。随着轮数 R 增加，最少活跃 S 盒数量快速上升，差分路径概率呈指数下降。后文将通过各轮数最少活跃 S 盒数量的评估结果进一步支持这一判断。

因此，本文给出的第二原像路径应理解为低轮约减模型中的结构性分析，而不能直接推广为对完整轮数 LLH 的有效攻击。对于更高轮数，扩散增强使局部可控差分难以维持；最少活跃 S 盒数量的快速增长又使任意精确差分路径的概率指数级下降。综合这两点，可以认为 LLH 在更高轮数下能够有效抵抗第二原像攻击。

4.5 抗碰撞分析

为评估 LLH 的抗碰撞安全性,从差分传播角度出发,核心在于判断是否存在高概率差分路径可用于构造碰撞。为此,采用自动化方法搜索最小活跃 S 盒差分路径,并据此评估碰撞攻击的安全性界。

基于 MILP 与 SAT 方法对差分传播过程进行统一建模与自动化搜索[6]-[7]。模型对 S 盒的差分传播进行比特级完全刻画,从而保证差分可达性描述的精确性;以最小化活跃 S 盒个数为优化目标,搜索给定轮数下的最优差分路径。

在直接碰撞搜索模型中,同时约束输入差分的比特率部分为非零、输出差分的比特率部分为零,并在差分路径搜索的基础上进一步完成差分填充。但受限于计算复杂度,该方法在实际中仅能支持 1 轮碰撞的搜索,结合线性化仿射子空间技术没有搜索到 2 轮及以上实际碰撞。

为进一步评估安全性,转而对置换本身建立最小活跃 S 盒下界模型,即直接分析差分传播过程中所需的最少活跃 S 盒个数。该方法可推进至前 3 轮,但随着轮数增加,模型规模迅速增长,计算复杂度显著提升,难以继续扩展。

在实际安全性评估中,采用限定搜索时间的策略,即在给定计算资源约束下,以当前可搜索到的最小活跃 S 盒个数作为近似最优结果,并与理论安全界进行比较。由于单个活跃 S 盒的最大差分概率为 2^{-2} ,若一条差分路径包含 N 个活跃 S 盒,则其概率上界为 $2^{-(2N)}$ 。为满足 d 比特的碰撞安全性要求,应满足 $N \geq d/2$ 。若在限定时间内搜索得到的最小活跃 S 盒个数达到或超过该阈值,则认为算法在实际意义下具备足够的抗碰撞能力。

实际搜索中，将 MILP 的搜索时间限定为 10 小时，并将当前搜索得到的最优结果记录于

表 4-4，其中前三轮结果为全局最优解。

表 4-4 各轮数活跃 S 盒数量评估结果

w=32							
#R	1	2	3	4	5	6	7
#S	1	4	10	24	58	192	266
w=64							
#R	1	2	3	4	5	6	7
#S	1	4	10	27	61	299	415
w=96							
#R	1	2	3	4	5	6	7
#S	1	4	10	27	61	376	601
w=128							
#R	1	2	3	4	5	6	7
#S	1	4	10	28	63	428	834

4.6 侧信道分析

4.6.1 侧信道分析对象

在传统的块密码（如 AES、SM4）侧信道分析中，攻击者的核心目标通常是恢复算法轮函数中使用的固定轮密钥[8]-[10]。然而，LLH 密码杂凑算法本身并不包含这种独立的密钥编排体系。尽管如此，在数字签名、消息认证码（MAC）生成以及密钥派生（KDF）等多种密码学实际应用场景中，该算法依然承载着保护敏感信息的重任。在这些特定场景下，输入消息 M 往往是由公开数据与攻击者未知的秘密信息（如用户口令、根密钥等）拼接而成。

因此，针对该哈希算法的侧信道攻击目标，必须且只能聚焦于恢复

输入的敏感消息 M 。根据该算法海绵结构 (Sponge Construction) 的变体设计, 输入消息经过规则填充后, 会被分割成多个长度为比特率 r 的消息块 M_i 。在极其关键的吸收阶段 (Absorbing Phase) 中, 这些消息块 M_i 会依次与内部状态的前 r 比特执行按位异或 (XOR) 操作。每完成一次异或, 系统就会紧接着执行一次复杂的置换函数 P 以混淆状态。这种“异或即置换”的机制导致融合了未知秘密消息的内部状态在硬件电路中产生大量数据翻转。对于侧信道分析而言, 这种不断吸收新鲜消息块的异或节点及其后续的初轮置换过程, 是物理侧信道泄露的攻击面。

4.6.2 平台正确性验证

实施硬件侧信道分析 (SCA) 的先决条件是将被测算法部署在 FPGA 或 ASIC 等物理载体上, 以获取真实的功耗或电磁泄露。因此, 我们首先需要完成算法的 RTL 级硬件设计。在此过程中, 选用经过逻辑验证的 C 语言实现版本 `llh_hash_optimize.c` 作为标准参考模型 (Golden Model), 用于生成标准的内部状态数据, 作为硬件测试的唯一对照基准。

本次硬件设计与侧信道评估聚焦于 LLH-512 算法实例。根据算法规范, 该实例的字宽配置为 $w=64$ 。算法的核心在于置换函数 P , 而该函数由多轮迭代的内部轮函数组成。为了实现精准的周期级控制与硬件资源评估, 我们完成了置换函数内部轮函数的硬件化开发, 即 Verilog 模块 `iRoundFunc_w64.v`。

在硬件仿真平台 (Testbench) 中, 我们设计了严格的逐字状态比对逻辑。仿真系统会在时钟上升沿持续监控模块的数据有效信号 (`out_valid`), 当检测到有效输出时, 将硬件模块计算得出的 64 比特宽数据段依次拼接到完整的状态寄存器 `got_state` 中。随后, 平台会自动将

拼接完成的硬件状态与由 `llh_hash_optimize.c` 提前计算并载入测试内存的期望状态矩 `expect_mem` 进行严格的等价判断。

验证逻辑的核心代码如下。若比对一致，系统将在控制台打印[PASS]及对应的测试用例编号；若出现数据错位或计算错误，则判定为[FAIL]标志，并同步打印出硬件的错误状态值与 C 语言模型的期望值以便追溯调试。

```
while (i < ROWS) begin
    @(posedge clk);
    if (out_valid) begin
        got_state[i*W+: W] = out_data;
        i = i + 1;
    end
end

if (got_state !== expect_mem[tc]) begin
    fail_cnt = fail_cnt + 1;
    $display("[FAIL] tc=%0d", tc);
    $display(" got  = %0384h", got_state);
    $display(" exp  = %0384h", expect_mem[tc]);
end else begin
    $display("[PASS] tc=%0d", tc);
end

if (!out_last)
```

在完成测试平台的搭建后，我们注入了多组边界与随机数据作为输入激励。如图 4-14 所示，全部 8 个独立测试用例的仿真结果均显示为[PASS]。这充分证明了 `iRoundFunc_w64.v` 模块在时序调度和数学功能上与标准软件行为完全一致。

```
[PASS] tc=1
[PASS] tc=2
[PASS] tc=3
[PASS] tc=4
[PASS] tc=5
[PASS] tc=6
[PASS] tc=7
ALL PASS (8/8)
```

图 4-14 测试结果

4.6.3 针对 24 比特超级 S 盒的模板攻击可行性分析

在明确了攻击目标后，需要寻找算法内部的非线性操作作为侧信道能量建模的突破口。深入剖析置换函数 P 的轮函数 RoundFun 可知，列替换（SubColumn）操作是整个轮函数中唯一的非线性变换步骤。该操作采用并行处理架构，对内部状态的 w 个切片列 C_j 进行处理，每个切片列都会被送入一个 24 比特的超级 S 盒 S_{24} 中进行变换。超级 S 盒 S_{24} 是一个将 24 比特输入映射为 24 比特输出的高维非线性置换。

现有最主流且最强大的侧信道分析手段是模板攻击（Profiling Attack）。模板攻击是一种基于多元高斯分布特征的分析方法，主要分为“模板构建”和“特征匹配”两个阶段。在构建阶段，攻击者需要在完全可控的测试设备上，针对目标数据的每一种可能输入，刻画其对应的功耗或电磁辐射概率分布模型，即计算均值向量和协方差矩阵。之所以要求每个模板最少需要采集 1000 条高质量曲线，是因为侧信道信号在物理收集中不可避免地会掺杂环境电子白噪声、示波器量化误差以及微小的时钟抖动。为了有效抵消这些随机噪声的干扰，并确保高维协方差矩阵在后续求逆运算中具备足够的数值稳定性与统计学显著性，必须依靠大批量重复采样来进行降噪。

然而，将建模攻击直接用于 LLH 超级 S 盒，从样本复杂度上是不可行的。具体的理论推演与硬件瓶颈分析如下：首先是理论建模的数据

规模发生爆炸。为了对超级 S 盒 S_{24} 建立具备全覆盖能力的功耗或电磁模型，理论上必须遍历其所有的输入可能性。由于输入位宽高达 24 比特，这就要求建立 2^{24} （即 16777216）个完全独立的特征模板。结合前文的统计学降噪要求，每个特征模板最小需要采集 1000 条高质量曲线，完成基础建模的总采样量将达到惊人的 1.67×10^{10} 条（即近乎 2^{34} 的量级，超百亿条物理曲线），不仅是样本复杂度过高，同时为分析时需要的存储空间带来无法克服的压力。

作为一种折中的处理方法，为避免建立 2^{24} 个全空间模板，可以利用汉明重量（Hamming Weight）进行侧信道能量建模。对于 24 比特的输入消息，其汉明重量仅有 0 到 24 共 25 种可能。若以此作为目标建立模板，总采样量将骤降至 25000 条（25 个模板，每个模板 1000 条曲线），这在现有的硬件采集平台上是完全可行的。

在输入消息随机均匀分布的前提下，24 比特消息的汉明重量出现概率服从二项分布。利用信息熵理论可以量化该建模方案下单次测量所能获取的平均还原信息量。在初始状态下，24 比特未知输入消息的初始信息熵为 24 比特。在侧信道测量获取汉明重量后，可利用基于正态分布的渐近近似公式计算平均剩余熵，即可得出平均还原信息量： $24 - 20.66 = 3.34$ 比特。

在针对该算法 24 比特超级 S 盒的侧信道分析中，若采用汉明重量建模方案，可以在较为合理的样本复杂度、分析复杂度水平下，从 24 比特消息中平均还原出约 3.34 比特的有效信息量。

总结而言，在硬件设计侧未引入任何侧信道防护机制的条件下，面对当前最强侧信道攻击，LLH 算法在理论上能达到最小泄漏量。未来，在面向侧信道攻击时，如需实现更高安全性，可以适当添加硬件级防护

设计。

4.7 抗量子计算分析

在量子计算环境下，输出长度为 d 比特的理想杂凑函数可由 Grover 搜索以约 $2^{d/2}$ 的复杂度进行原像搜索[11]-[13]。LLH 算法输出长度为 $d=8w$ ，因此 LLH-256、LLH-512、LLH-768 和 LLH-1024 的通用量子原像安全目标分别为 2^{128} 、 2^{256} 、 2^{384} 和 2^{512} 。

下面具体讨论单向性情况，即随机选择消息 M ，计算其杂凑值 $H(M)$ 后，攻击者给出满足同一杂凑值的其它合法原像 M' 是困难的，即 $H(M') = H(M)$ 。在量子模型下，攻击者允许以叠加方式查询底层置换。对 LLH 算法前馈海绵结构变体模式，在量子随机置换模型下可以将单块消息情形转化为单轮迭代。具体地，单块消息吸收满足

$$S^{(1)} = P(S^{(0)}) \oplus (M \parallel S_{r:b-1}^{(0)}).$$

其中 $P(S^{(0)})$ 为已知常量，记为 (U, V) 。则输出杂凑值可写为

$$H_P^{(1)}(M) = R\left(P\left(U \oplus M \parallel V \oplus S_{r:b-1}^{(0)}\right)\right),$$

其中 R 为取状态的比特率部分。由相应量子单向性结果，可得进行至多 q 次量子查询的攻击者成功概率满足

$$\Pr\left[OW_{\mathcal{A}}^{(1)}(q) = 1\right] \leq o\left(\frac{q^2}{2^r}\right) + o\left(\frac{q^2}{2^c}\right).$$

多块情形可通过 hybrid 方法逐层处理，得到形式上

$$\Pr\left[OW_{\mathcal{A}}^{(1)}(q) = 1\right] \leq o\left(\frac{q^2}{2^r}\right) + o\left(\frac{mq^2}{2^c}\right).$$

由于 LLH 中 $r = 8w$ 、 $c = 16w$ ，主导项为 $O(q^2/2^{8w})$ ，对应量子原

像安全量级为 2^{4w} ；容量相关坏事件项由 $c=16w$ 控制，不主导上述界。

在具体轮函数层面，量子分析沿用经典原像分析得到的缩减轮攻击候选。单块两轮和双块极低轮模型已经能被经典 SAT 求解，因此属于低轮经典失守模型，不需要量子加速。单块三轮的主要候选是 $6w$ 部分线性化路线：经典外层需要枚举约 2^{6w} 个固定模式，若只对这一外层枚举做量子幅度放大，理想情况下可降为 2^{3w} 。对 $w = 32$ ，该外层量级为 2^{96} ，形式上低于 LLH-256 的通用量子基线 2^{128} 。但是，该估计还不是完整量子攻击复杂度，因为每次外层模式命中后仍需要调用局部验证器。前述“正确 $6w$ 模式已知”的 CNF 实验显示，即使正确模式已知，Kissat 在限定时间内仍未返回 SAT 或 UNSAT，说明局部验证器尚未被降到低成本水平。因此，单块三轮目前只能视为有研究价值的量子化候选，而非完整攻击。

对单块四轮，当前部分线性化路线的经典外层规模不低于 2^{12w} ，即使做理想量子幅度放大也不低于 2^{6w} ，仍高于通用 Grover 基线 2^{4w} 。标准轮数 16、24、32、40 距离该缩减轮候选更远。因此，当前未发现可延伸到标准轮数并优于 Grover 基线的专用量子原像攻击。结合模式级单向性分析，LLH-256、LLH-512、LLH-768 和 LLH-1024 在量子随机置换模型下分别达到 2^{128} 、 2^{256} 、 2^{384} 和 2^{512} 的通用量子原像安全量级。

5 算法随机性统计测试

为评估 LLH 杂凑算法输出序列的统计随机性与扩散特性，本文对四个标准实例 LLH-256、LLH-512、LLH-768 和 LLH-1024 进行了系统的经验统计测试。测试工作分为三部分：其一，对输出序列进行基础统计，考察比特频率平衡性与字节分布均匀性；其二，采用 NIST SP 800-

22 Statistical Test Suite 对输出序列进行标准随机性检验；其三，针对输入单比特扰动开展雪崩效应测试，以评估算法的局部扩散能力。通过上述三类测试，可以从不同角度考察 LLH 输出是否呈现接近理想随机序列的统计行为。

5.1 测试对象与测试设置

测试对象为 LLH 的四个标准输出版本，即 LLH-256、LLH-512、LLH-768 和 LLH-1024。对每个版本，分别采用两种输入构造方式生成测试序列：

1. 顺序输入方式（sequential）：按顺序构造输入消息；
2. 随机输入方式（random）：随机生成输入消息。

对于每一种“版本 - 输入方式”组合，均生成 55 条独立二进制序列，每条序列长度为 10^6 比特，并将其作为正式随机性测试输入。因此，正式 NIST 统计测试共覆盖 8 组数据：

- LLH-256 / sequential
- LLH-256 / random
- LLH-512 / sequential
- LLH-512 / random
- LLH-768 / sequential
- LLH-768 / random
- LLH-1024 / sequential
- LLH-1024 / random

在此基础上，对每组输出序列进一步统计比特“1”比例与字节级 Shannon 熵，并对四个版本分别在 16 字节、32 字节和 64 字节输入长度下进行雪崩效应测试。雪崩测试中，每组采用 1000 个随机样本，对输入

逐位翻转后统计输出比特翻转比例。

5.2 基础统计结果

首先，对正式测试中生成的 8 组输出序列进行了基础统计分析。结果表明，8 组序列的比特“1”比例均非常接近理想值 0.5，其取值范围为

$$0.49996116 \sim 0.50005864.$$

这说明 LLH 的输出在整体频率意义上不存在显著的偏零或偏一现象。

同时，对输出序列进行了字节级 Shannon 熵估计。8 组正式测试序列的字节熵取值范围为

$$7.99849610 \sim 7.99854584 \text{ bits/byte},$$

与理想均匀分布对应的 8 bits/byte 非常接近。该结果表明，从字节分布角度看，LLH 的输出序列未呈现明显的统计失衡。

因此，在最基本的频率平衡性和分布均匀性层面，LLH 四个标准版本在两种输入构造方式下均表现出良好的统计随机性外观。

5.3 NIST SP 800-22 统计测试结果

在基础统计分析之后，本文采用 NIST SP 800-22 Statistical Test Suite 对上述 8 组正式测试序列进行了标准随机性检验。所采用的测试项目包括：

- Frequency;
- Block Frequency;
- Cumulative Sums;
- Runs;
- Longest Run of Ones;

- Rank;
- Discrete Fourier Transform;
- NonOverlapping Template Matching;
- Overlapping Template Matching;
- Universal;
- Approximate Entropy;
- Random Excursions;
- Random Excursions Variant;
- Serial;
- Linear Complexity。

对样本数为 55 的测试项，NIST 报告给出的最低参考通过率约为 (52/55)；对 Random Excursions 及 Random Excursions Variant，由于有效样本数为 44，其最低参考通过率约为(41/44)。

测试结果表明，8 组正式测试均成功完成全部 NIST SP 800-22 检验流程，均能够生成对应的`finalAnalysisReport.txt`。从汇总结果看，Frequency、Block Frequency、Runs、Longest Run、Rank、FFT、Universal、Approximate Entropy、Serial、Linear Complexity 等基础测试项目在各版本及两种输入构造方式下整体表现稳定，未观察到跨版本、跨输入方式重复出现的系统性异常现象。

以 LLH-256 在 sequential 输入方式下的代表性结果为例，其主要测试项通过情况如下：

- Frequency: (54/55);
- Block Frequency: (55/55);
- Cumulative Sums: (53/55) 和 (52/55);

- Runs: (55/55);
- Longest Run: (54/55);
- Rank: (54/55);
- FFT: (54/55);
- Serial: (54/55) 和 (55/55);
- Linear Complexity: (54/55)。

上述项目均达到相应参考门槛。对于 Random Excursions 与 Random Excursions Variant，由于有效样本数为 44，其各子项通过比例均达到或超过(41/44)的参考门槛，因而也未表现出显著异常。

在代表性报告中，仅观察到 NonOverlapping Template Matching 的个别子测试项出现轻微未达标现象。例如，在 LLH-256 的 sequential 输入方式下，有一个子项的通过比例为(51/55)，略低于(52/55)的参考门槛。考虑到模板类测试本身包含大量子项，其结果通常具有较高离散性，个别子项出现局部轻微波动属于经验统计测试中可以接受的现象。更重要的是，此类波动未在多个版本和不同输入方式下形成一致、重复的异常模式，也未伴随基础测试项的大面积失效。因此，从整体上看，未发现支持 LLH 输出存在显著系统性统计偏差的证据。

综合全部正式 NIST 检验结果，可以认为 LLH 在统计随机性意义上整体表现良好。

5.4 雪崩效应测试结果

为进一步考察 LLH 的扩散能力，本文对四个标准版本分别在 16 字节、32 字节和 64 字节输入长度下进行了雪崩效应测试。对每一组输入长度，随机选取 1000 个样本，对输入逐位翻转后统计输出比特翻转比

例。

正式汇总结果显示，12 组雪崩测试的平均翻转比例范围为

0.49989435~0.50009436,

均与理想值 0.5 高度一致。这表明，当输入仅发生单比特扰动时，LLH 输出端能够在统计平均意义下产生近似一半输出比特翻转的效果，符合杂凑函数对扩散性的基本预期。

对应的翻转比例标准差范围为

0.01559049~0.03127503.

从整体趋势看，随着输出长度的增加，翻转比例的标准差总体呈下降趋势。这说明较长输出版本在统计平均意义下具有更稳定的扩散集中性。与此同时，在 16 字节、32 字节和 64 字节三种输入长度下，四个版本的平均翻转比例均保持在极接近 0.5 的范围内，说明 LLH 的扩散能力对输入长度变化不敏感，具有较好的稳定性。

因此，雪崩测试结果表明，LLH 在所有标准版本上均具备良好的局部扩散能力。

5.5 综合结论

根据基础统计分析、NIST SP 800-22 标准随机性检验以及雪崩效应测试，可以得到如下结论。

首先，在基础统计层面，LLH 四个标准版本在 sequential 与 random 两种输入构造方式下，其输出序列的比特“1”比例均高度接近 0.5，字节级 Shannon 熵均高度接近理想值 8 bits/byte，表明输出在整体频率平衡性和字节分布均匀性方面表现良好。

其次，在 NIST SP 800-22 正式检验中，8 组测试均成功完成全部标准测试流程。各组在 Frequency、Block Frequency、Runs、Longest Run、

Rank、FFT、Universal、Approximate Entropy、Serial、Linear Complexity 等基础项目上均表现稳定，未出现跨版本、跨输入方式的一致性异常。尽管在模板类测试的个别子项中观察到轻微局部波动，但未形成系统性失效模式，因此整体结果支持 LLH 输出具有良好的统计随机性。

最后，在雪崩效应测试中，所有标准版本在多种输入长度下的平均输出翻转比例均非常接近 0.5，表明 LLH 对输入单比特扰动具有良好的扩散能力。

综上所述，在本次测试所采用的样本规模、测试方法与参数设置下，LLH 杂凑算法四个标准版本均未表现出显著的统计随机性异常，其输出序列整体上具有良好的随机性统计特征。

6 软硬件性能评估

6.1 软件性能评估

6.1.1 Intel/Amd 平台

本节面向 Intel/AMD 通用 x86-64 平台，对 LLH 四个标准实例的参考实现版与性能优化版实现进行软件性能评估。本次实现按照《新一代商用密码算法 x86 架构实现自评估指引》的要求，完成了面向 AVX2 路径的性能优化，并按正式提交口径进行测试与结果整理。

本节测试用例与《新一代商用密码算法 x86 架构实现自评估指引》保持一致，采用密码杂凑算法规定的 S1-S8 八级输入长度，即 32B、128B、512B、1024B、4096B、8192B、16384B、65536B。

6.1.1.1 测试环境及用例

测试平台采用 Intel x86-64 物理机。为保证结果的可复核性，测试时统一固定编译器、构建工具、编译参数和 CPU 绑定方式，并在同一

逻辑核上完成正式测量。本节优化实现按照 AVX2 口径完成编译和执行。

表 6-1 Intel/AMD 平台测试环境

项目	规格
平台类型	物理机
CPU	Intel(R) Xeon(R) Gold 6258R CPU @ 2.70GHz
内存	503 GiB
SIMD 能力	AVX2
操作系统	Ubuntu 22.04.5 LTS
内核版本	Linux 6.8.0-107-generic x86_64 GNU/Linux
编译器	GCC 12.3.0
构建工具	CMake 3.28.3

测试对象包括 LLH-256、LLH-512、LLH-768、LLH-1024 四个标准实例的参考实现版与性能优化版。测试采用如下用例组织方式：

1. 功能测试：在表 6-2 八个长度点上计算参考实现版和性能优化版的杂凑值，比较结果是否完全一致。
2. 性能测试：对表 6-2 八个长度点分别运行 100 次，测量时钟周期数和吞吐率，并取平均值。其中，平均周期数用于反映单次调用的核心计算开销，平均吞吐率用于反映持续处理数据时的软件处理能力。
3. 输入生成：使用固定种子标签 KAT_2_12 生成指定长度的随机消息，保证功能测试和性能测试数据可复现。

表 6-2 Intel/AMD 平台测试用例

级别	数据长度
S1	32 Bytes
S2	128 Bytes
S3	512 Bytes
S4	1024 Bytes
S5	4096 Bytes
S6	8192 Bytes
S7	16384 Bytes
S8	65536 Bytes

6.1.1.2 部分针对性优化技巧

本节选取优化实现中若干具有代表性的、面向 Intel/AMD x86-64 平台 AVX2 路径的实现技巧进行说明。相关优化保持 LLH 算法定义、输入输出语义及测试向量结果一致，其目标是在不改变算法行为的前提下提高通用 CPU 平台上的软件执行效率。

首先，针对 $w=32/64/96/128$ 四种标准字宽分别生成实例化实现。该方式避免了运行期的通用参数分派，使轮常数、循环移位参数、状态装载与存储路径以及轮函数内部的数据通路均在编译期固化，从而有利于编译器完成常量传播、循环展开、指令选择和寄存器分配。对于字宽不同而控制结构相同的杂凑实例，这种按实例展开的实现方式能够减少通用代码引入的额外控制开销，并提高核心轮函数的指令局部性。

其次，优化实现面向 AVX2 向量指令通路，对核心轮函数中的布尔运算、字级循环移位和状态重排过程进行了并行化组织。LLH 轮函数由大量按位异或、按位与、按位取反及固定移位组成，这类操作具有较强的数据并发特征，适合映射到 AVX2 指令上执行。优化实现据此对状态

更新过程进行向量化设计，以降低单位轮次的指令条数和访存压力，并提高多个状态分量同时更新时的吞吐能力。

再次，针对字节对齐输入，优化实现采用了专门的快速处理路径。接口层首先依据消息比特长度是否为 8 的整数倍进行分流：对于字节对齐消息，直接进入面向整字节输入的专用处理流程；仅当末尾存在不足 1 字节的比特串时，才保留按位掩码与逐步收尾的通用处理方式。该机制减少了短消息场景中的缓冲管理、边界判断和控制转移开销，因此对 S1-S4 区间的时延改善尤为明显。

此外，针对完整消息块与尾块，优化实现采用分离的处理机制。对于完整分组输入，在当前处理位置已经与分组边界对齐时，直接按整分组连续处理；对于末尾不足一个分组的数据，则单独构造尾块并完成最终状态变换与结果提取。对于长度不超过一个分组的短消息，进一步绕过通用吸收流程，直接进入尾块处理与输出阶段。该分流机制减少了缓冲区维护、数据搬移和边界判断开销，从而同时改善了短消息时延和长消息的连续处理效率。

最后，为适应现代服务器处理器的多级缓存和乱序执行特征，优化实现还对状态数据布局、临时变量复用和关键路径上的访存次数进行了约束。其基本原则是尽量保持热点状态驻留在寄存器或一级缓存可高效访问的范围内，避免不必要的中间结果回写与重复装载，从而在算术逻辑运算与数据访存之间获得更好的平衡。

综上，本轮 AVX2 优化实现的性能收益主要来源于实例特化、AVX2 向量并行化、短消息快速处理、块级处理拆分以及状态数据组织优化。

6.1.1.3 测试结果数据

(1) 功能一致性

正式测试中，参考实现版与性能优化版在四个实例上的测试向量结果完全一致，表明 AVX2 优化路径未改变算法功能语义。

(2) 参考实现版与性能优化版总体对比

表 6-3 给出四个 LLH 实例在代表性长消息场景 S8 (65536B) 下的参考实现版与性能优化版对比。可以看到，优化实现相对于参考实现均获得了稳定且明确的改进。

表 6-3 S8 场景下性能优化版与参考实现版对比摘要

实例	参考实现版	性能优化版	周期数	参考实现版	性能优化版	吞吐率
	平均周期数	平均周期数	优化倍数	吞吐率(MB/s)	吞吐率(MB/s)	提升倍数
LLH-256	5607558.98	713344.98	7.86x	31.48	245.23	7.79x
LLH-512	4121539.80	655538.76	6.29x	42.79	267.64	6.25x
LLH-768	6970655.82	1603857.26	4.35x	25.21	110.26	4.37x
LLH-1024	5631252.02	1135454.8	4.96x	31.18	154.47	4.95x

对 S1-S8 八个长度点整体观察，性能优化版的平均收益如下。

表 6-4 S1-S8 八个长度点的平均优化收益

实例	周期数	吞吐率
	优化倍数	提升倍数
LLH-256	7.56x	7.71x
LLH-512	6.19x	6.20x
LLH-768	4.23x	4.21x
LLH-1024	4.86x	4.88x

从总体结果看，LLH-256 与 LLH-512 的相对优化收益更高，LLH-

768 与 LLH-1024 也保持了显著提升但增幅相对较小。结合实现结构可以看出，较宽状态实例在轮内需要处理更多状态字和数据重排，因此即便在 AVX2 路径下仍能得到明显收益，其相对加速比也通常低于较窄状态实例。

(3) S1-S8 详细结果

表 6-5 给出四个实例在 S1-S8 八个长度点上的详细结果。表中同时列出参考实现版与性能优化版的平均周期数和平均吞吐率。

表 6-5 Intel/AMD 平台 S1-S8 详细性能结果

实例	数据长度(B)	参考实现版 平均周期数	性能优化版 平均周期数	参考实现版 吞吐率(MB/s)	性能优化版 吞吐率(MB/s)
LLH-256	32	5201.04	728.64	16.6	123.9
LLH-256	128	13229.54	1845.98	25.88	198.58
LLH-256	512	45609.48	5945.76	30.09	232.99
LLH-256	1024	89157	11558.86	30.97	241
LLH-256	4096	352400.82	47397.08	31.11	244.7
LLH-256	8192	697555.44	89903.6	31.71	242.48
LLH-256	16384	1389861.58	180367.22	31.61	244.8
LLH-256	65536	5607558.98	713344.98	31.48	245.23
LLH-512	32	7774.86	1322.2	11.2	66.53
LLH-512	128	11817.56	1902.1	29.04	173.08
LLH-512	512	37278.86	5759.18	38.25	239.73
LLH-512	1024	68813.56	11264.18	39.8	253.4
LLH-512	4096	260875.42	42282.64	41.94	264.91
LLH-512	8192	515594.76	83178.72	42.4	266.7
LLH-512	16384	1032172.62	167250.56	42.82	265.51
LLH-512	65536	4121539.8	655538.76	42.79	267.64
LLH-768	32	19737.36	4919.18	4.35	16.77
LLH-768	128	31162.24	7617.72	11.78	49.39
LLH-768	512	71905.5	17222	19.37	84.25
LLH-768	1024	123043.3	28825.82	22.81	92.69
LLH-768	4096	452046.76	104917.72	24.29	101.73
LLH-768	8192	886817.54	207305.88	24.68	106.42
LLH-768	16384	1776914.9	408400.94	24.76	108.35
LLH-768	65536	6970655.82	1603857.26	25.21	110.26

LLH-1024	32	22242.32	4629.5	4.06	18.72
LLH-1024	128	21811.84	4678.36	15.89	73.85
LLH-1024	512	55382.04	11333.3	25	122.92
LLH-1024	1024	99417.66	20200.16	27.41	137.16
LLH-1024	4096	365359.66	74510.38	30.05	149.84
LLH-1024	8192	710877.16	145595.14	30.7	152.28
LLH-1024	16384	1414198.56	288721.38	30.83	152.15
LLH-1024	65536	5631252.02	1135454.8	31.18	154.47

从表 6-5 可以看到，性能优化版的吞吐率在全部实例和全部长度点上均高于参考实现版，平均周期数在全部长度点上均显著低于参考实现版。这说明本轮 AVX2 路径优化在短消息和长消息场景下都具备较好的稳定性。

6.1.1.4 测试结论

综合测试结果可以得到以下结论。

第一，LLH 四个标准实例在 Intel/AMD 通用 x86-64 平台上均实现了稳定而明确的 AVX2 软件优化收益。相对于参考实现版，性能优化版不仅在 S1-S4 短消息区间显著降低了单次调用开销，而且在 S5-S8 较长消息区间也保持了持续稳定的加速。

第二，从代表性长消息场景 S8 的结果看，LLH-256、LLH-512、LLH-768、LLH-1024 的平均周期数优化倍数分别达到 7.86x、6.29x、4.35x、4.96x；对应吞吐率提升倍数分别达到 7.79x、6.25x、4.37x、4.95x。可见 AVX2 路径已经能够对四个实例提供实质性的性能改善，而不仅是局部微调。

第三，从 S1-S8 八个长度点的平均结果看，LLH-256 和 LLH-512 的收益更为突出，LLH-768 与 LLH-1024 也保持了 4 倍以上的平均周期数优化。这表明 LLH 的轮函数和数据路径具有较好的 SIMD 向量化适配性，性能优化版能够在保持算法语义、接口行为和测试向量结果

一致的前提下显著改善软件执行效率。

总体而言，本轮结果表明，LLH 在 Intel/AMD 通用 x86-64 平台上具有良好的 AVX2 软件实现可优化性。

6.1.1.5 AVX-512 优化路径补充说明

除前述 AVX2 优化实现外，LLH 在同一测试环境开展了面向更宽向量指令通路的性能优化测试。该路径同样保持 LLH 算法定义、输入输出语义及测试向量结果一致，其优化思路主要包括：利用 AVX-512 指令集，实现更宽向量寄存器下的状态并行组织、布尔运算与重排操作的向量化收敛，以及短消息与整块处理路径上的指令级压缩。以下结果用于补充展示 LLH 在 AVX-512 条件下的软件优化效果，供支持 AVX-512 的平台参考。本节提到的“优化实现”均指利用 AVX-512 指令能力实现的性能优化版本。

测试对象包括 LLH-256、LLH-512、LLH-768、LLH-1024 四个标准实例的参考实现与 AVX-512 优化实现。为评估相同输出长度条件下的软件效率，本节进一步引入以下基线算法：

- LLH-256 对比 SHA-256
- LLH-512 对比 SHA-512
- LLH-768 对比 SHAKE256-768
- LLH-1024 对比 SHAKE256-1024

其中，SHA-256、SHA-512、SHAKE256-768 和 SHAKE256-1024 的软件实现结果均来自 OpenSSL 3.0.2。需要说明的是，后续所有基线比较仅用于相同输出长度条件下的软件效率分析，不构成密码学安全性意义上的等价比较。

本节采用两类测试结果。第一类结果（Test1~4）用于反映若干典型

消息规模下的总体性能，第二类结果（Test5）用于刻画短消息区间内部随输入长度变化的细粒度性能特征。前者采用 100 次重复测量并取均值，后者对每个输入长度点独立测量 30 次并取均值。

表 6-6 Intel/AMD 平台 AVX-512 优化版测试用例

编号	数据规模	含义
Test1	1B~512B 短消息集合	将 1B 至 512B 的消息各处理一次，将整个短消息区间作为一个整体给出结果
Test2	1KB	给出 1KB 消息的性能结果
Test3	1MB	给出 1MB 消息的性能结果
Test4	1GB	给出 1GB 消息的性能结果
Test5	0B~512B	给出 0B 至 512B 每一个输入长度各自的性能结果

在指标定义上，本节采用执行时间、吞吐率、优化加速比和相对基线差异四类指标。需要特别区分的是：Test1 反映短消息区间整体处理能力，可与 1KB、1MB、1GB 三类结果并列比较；Test5 则用于描述 0B~512B 区间内不同输入长度点的局部性能变化，二者对应的观察对象不同。

（1）参考实现与优化实现总体对比

表 6-7 汇总了四个 LLH 实例在 Intel/AMD 平台上的参考实现与优化实现性能结果。总体来看，优化实现相对于参考实现均获得了稳定的性能增益。

表 6-7 AVX-512 优化实现与参考实现对比摘要

实例	数据组	参考实现时延 (ms)	优化实现时延 (ms)	参考实现吞吐 (MiB/s)	优化实现吞吐 (MiB/s)	优化加速 比
LLH-256	1~512B	3.615436	0.442468	34.685116	283.464505	8.171068
LLH-256	1KB	0.024437	0.002921	40.004304	334.93651	8.365971
LLH-256	1MB	24.146783	2.83776	41.442178	352.603499	8.5091
LLH-256	1GB	24769.77576	2897.485173	41.341017	353.411662	8.548715
LLH-512	1~512B	2.286199	0.56015	54.789275	223.654649	4.081405

LLH-512	1KB	0.014116	0.003259	70.073789	305.212601	4.33139
LLH-512	1MB	13.363578	3.063043	74.836982	326.507711	4.362844
LLH-512	1GB	13738.38189	3136.134572	74.538194	326.51841	4.380674
LLH-768	1~512B	7.609766	1.450457	16.459571	86.36024	5.246461
LLH-768	1KB	0.042873	0.008033	22.781962	121.583493	5.337109
LLH-768	1MB	39.018024	7.305698	25.631864	136.892905	5.340766
LLH-768	1GB	40071.19708	7475.691516	25.554606	136.978336	5.360199
LLH-1024	1~512B	6.012693	1.156127	20.835207	108.358077	5.20072
LLH-1024	1KB	0.030114	0.006017	32.641641	167.183567	5.00482
LLH-1024	1MB	27.337303	5.259274	36.586926	190.196544	5.197923
LLH-1024	1GB	29669.99969	5408.945536	34.513521	189.317395	5.485357

图 6-1 给出四个 LLH 实例在四类消息规模下的吞吐率结果。可以看到，AVX-512 优化实现（opt）的吞吐率在各实例和各消息规模下均明显高于参考实现（ref）。

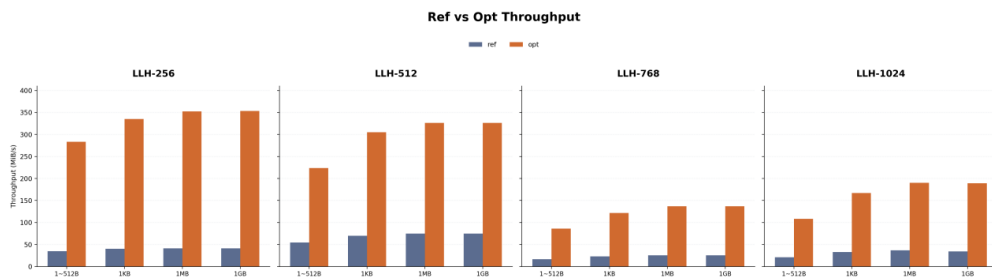


图 6-1 各实例参考实现与 AVX-512 优化实现吞吐率对比

图 6-2 以执行时间为指标展示参考实现与优化实现的差异。优化实现在各实例上的时延均显著低于参考实现。

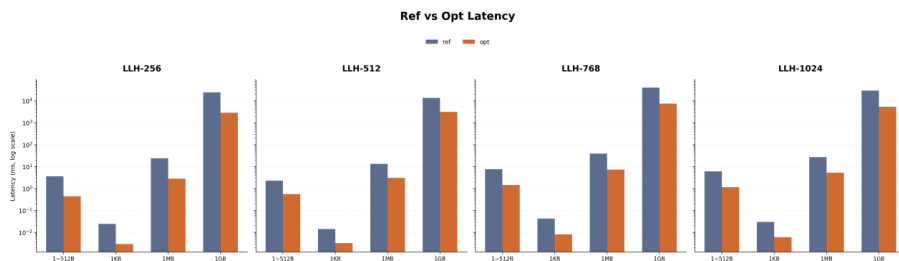


图 6-2 各实例参考实现与 AVX-512 优化实现时延对比

图 6-3 给出了各实例在不同消息规模下的优化加速比。结果表明，同一实例在不同消息规模下的收益显著且波动相对有限，说明优化效果

具有较好的稳定性。

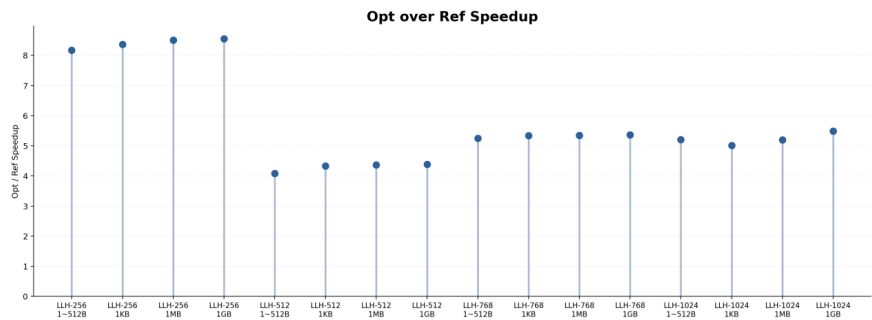


图 6-3 各实例 AVX-512 优化实现相对于参考实现的加速比

(2) 短消息区间细粒度结果

图 6-4 展示短消息区间内吞吐率随输入长度的变化。整体而言，AVX-512 优化实现的吞吐率曲线位于参考实现之上。

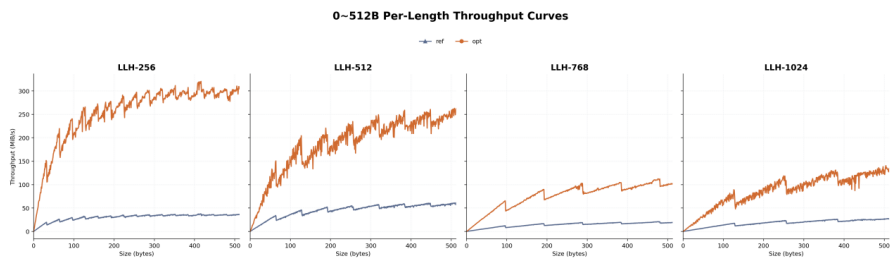


图 6-4 0B~512B 各输入长度吞吐率曲线

图 6-5 以对数坐标展示短消息区间内时延变化。整体而言，AVX-512 优化实现的时延曲线位于参考实现之下。

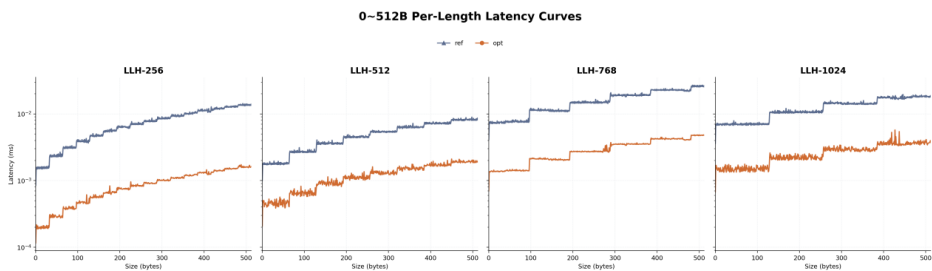


图 6-5 0B~512B 各输入长度时延曲线

(3) 与基线算法的对比

表 6-8 汇总了 AVX-512 优化实现相对于基线算法的软件效率位置。

可以看到，不同实例在不同消息规模下的相对优势并不相同。

表 6-8 AVX-512 优化实现与基线算法对比摘要

实例	数据组	优化实现时延 (ms)	基线算法时延 (ms)	优化实现吞吐 (MiB/s)	基线算法吞吐 (MiB/s)	优化实现 相对基线
LLH-256	1~512B	0.442468	0.583416	283.464505	216.180759	31.12%
LLH-256	1KB	0.002921	0.003694	334.93651	264.465041	26.65%
LLH-256	1MB	2.83776	2.138659	352.603499	468.234064	-24.70%
LLH-256	1GB	2897.485173	2210.407338	353.411662	463.330884	-23.72%
LLH-512	1~512B	0.56015	0.480522	223.654649	260.858491	-14.26%
LLH-512	1KB	0.003259	0.0061	305.212601	160.721032	89.90%
LLH-512	1MB	3.063043	1.390003	326.507711	719.530901	-54.62%
LLH-512	1GB	3136.134572	1441.571258	326.51841	710.360735	-54.03%
LLH-768	1~512B	1.450457	0.680698	86.36024	184.029642	-53.07%
LLH-768	1KB	0.008033	0.003326	121.583493	294.91693	-58.77%
LLH-768	1MB	7.305698	2.629677	136.892905	380.336257	-64.01%
LLH-768	1GB	7475.691516	2708.369742	136.978336	378.093105	-63.77%
LLH-1024	1~512B	1.156127	0.654605	108.358077	191.423142	-43.39%
LLH-1024	1KB	0.006017	0.009446	167.183567	103.505808	61.52%
LLH-1024	1MB	5.259274	2.589454	190.196544	386.387013	-50.78%
LLH-1024	1GB	5408.945536	2762.050165	189.317395	370.741501	-48.94%

图 6-6 给出 LLH-256、LLH-512 与对应 SHA-256、SHA-512 基线的吞吐率对比，可见 LLH-256 在短消息与 1KB 情形下占优，而 LLH-512 仅在 1KB 处高于基线。

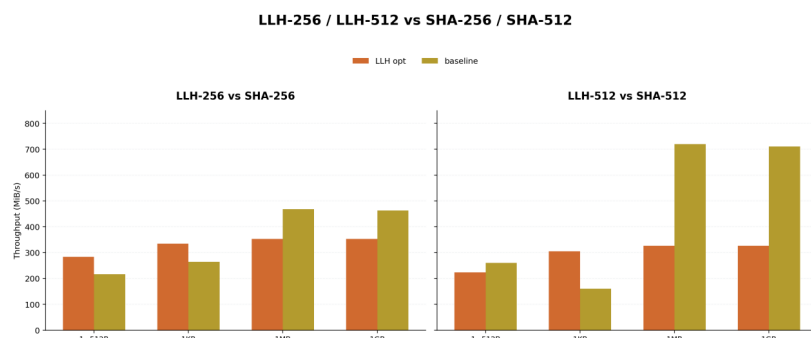


图 6-6 LLH-256/LLH-512 与 SHA-256/SHA-512 吞吐率对比

图 6-7 展示 1B~512B 各输入长度下优化实现相对 SHA-256 与 SHA-512 的吞吐率差异百分比,可见 LLH-256 多数长度点高于基线,而 LLH-512 的相对位置随输入长度变化更为明显,但总体优于基线。本图横轴取 1B~512B,而未包含 0B。原因在于,本图展示的是优化实现相对基线算法的吞吐率差异百分比;在 0B 输入下,优化实现与基线算法的吞吐率均为 0,因此相对差异为未定义量,不能纳入同一百分比口径下比较。

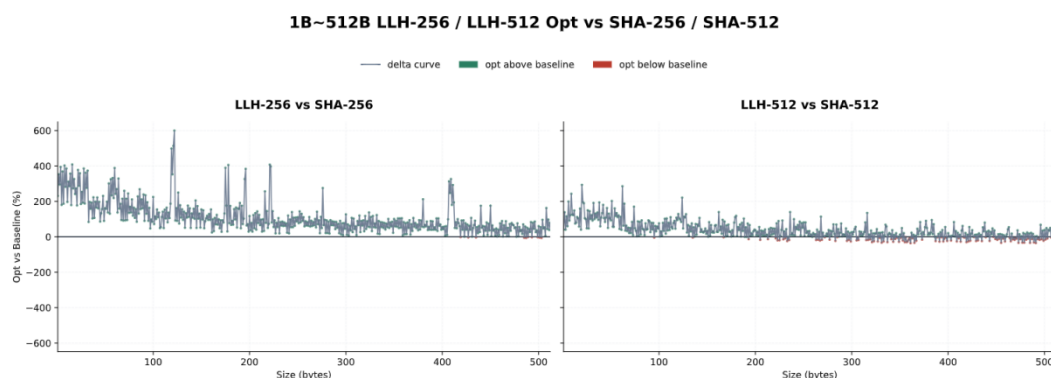


图 6-7 1B~512B 各输入长度下 LLH-256 和 LLH-512 优化实现
与基线算法的吞吐率差异

图 6-8 给出 LLH-768、LLH-1024 与对应 SHAKE256 基线的吞吐率对比,可见 LLH-768 在四类消息规模下均低于基线,而 LLH-1024 仅在 1KB 处高于基线。

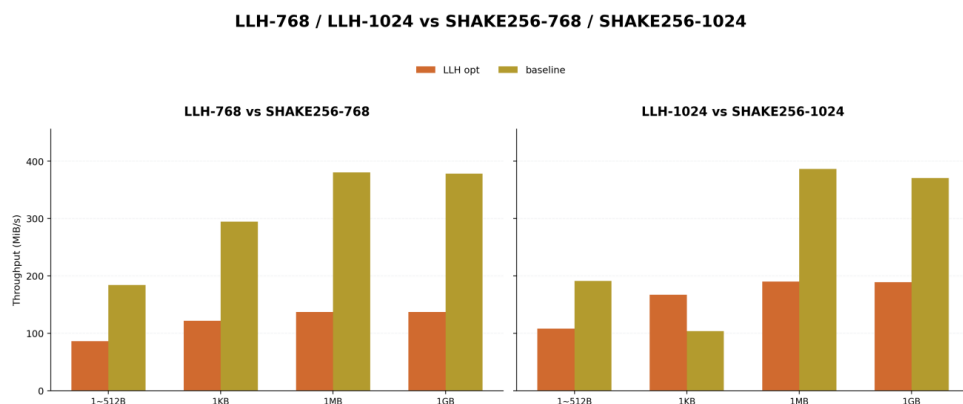


图 6-8 LLH-768/LLH-1024 与 SHAKE256-768/SHAKE256-1024 吞吐率对比

图 6-9 展示 1B~512B 各输入长度下优化实现相对 SHAKE256-768 与 SHAKE256-1024 的吞吐率差异百分比，可见 LLH-768 以负差异为主，LLH-1024 仅在部分长度区间接近或超过基线。本图横轴取 1B~512B，而未包含 0B。原因在于，在 0B 输入下，优化实现与基线算法的吞吐率均为 0，因此相对差异为未定义量，不能纳入同一百分比口径下比较。

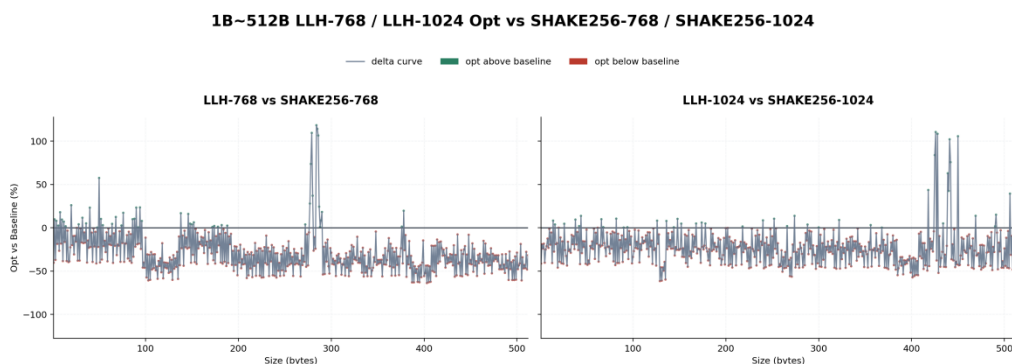


图 6-9 1B~512B 各输入长度下 LLH-768 与 LLH-1024 优化实现 相对基线算法的吞吐率差异

图 6-10 以百分比形式汇总优化实现相对基线算法的吞吐率差异，可见正增益主要集中在 LLH-256 的短消息与 1KB、LLH-512 的 1KB，以及 LLH-1024 的 1KB。

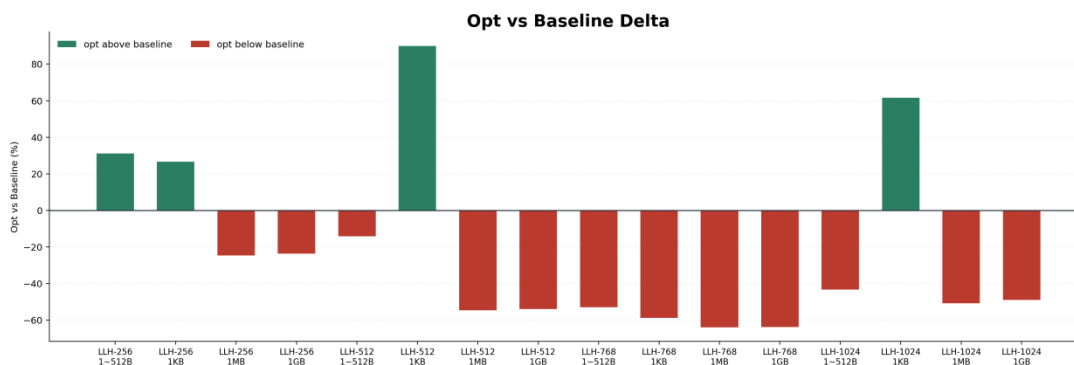


图 6-10 优化实现相对基线算法的吞吐率百分比差异

图 6-11 同时给出参考实现、优化实现与基线算法的吞吐率结果，可见优化实现相对于参考实现均明显提升，但相对于基线算法的优势随实例和消息规模而变化。

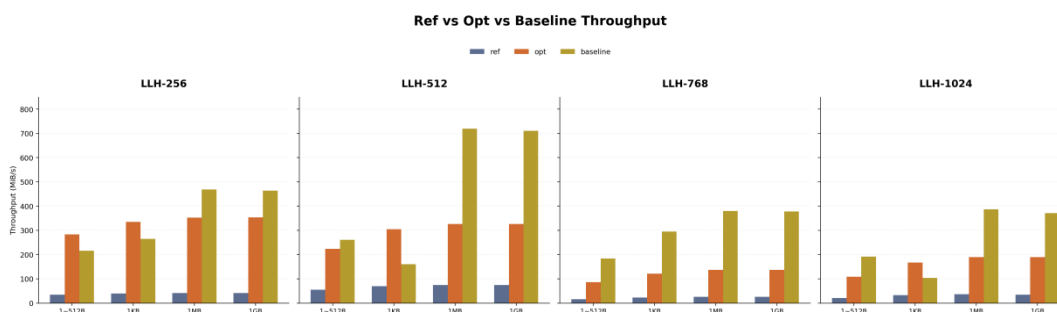


图 6-11 各实例参考实现、优化实现与基线算法吞吐率对比

需要说明的是，本节所有基线比较均采用“相同输出长度”下的软件效率对比方式，即 LLH-256 对比 SHA-256，LLH-512 对比 SHA-512，LLH-768 对比 SHAKE256-768，LLH-1024 对比 SHAKE256-1024。该比较仅反映相同输出长度条件下的软件实现效率差异。由于这些算法的安全强度并不一致，因此该对比不具备密码学意义上的等价性，不能据此推出算法整体优劣结论。

6.1.2 Arm 平台

本节面向多核并行场景，对 LLH 四个标准实例 LLH-256、LLH-512、

LLH-768 与 LLH-1024（分别对应字宽 $w=32、64、96、128$ ）的单线程实现与多线程并行实现进行软件性能评估。与 6.1.1 节在单一服务器平台上比较参考实现与优化实现的目标不同，本节的评估重点在于考察并行化策略在不同硬件环境下的可扩展性与实际收益。为此，测试在一台通用桌面级处理器（Intel Core i7）与一块以 ARM 架构为核心的嵌入式平台（树莓派）上同时开展，以便在算力、缓存容量和内存带宽差异显著的两类环境中，量化多线程并行带来的吞吐量提升与加速比，并分析线程创建、调度与核间迁移等开销对结果的影响。

为区分不同的并行化策略，本节给出三种实现版本。三者的算法定义、接口语义、长度编码、填充规则以及吸收与挤压流程均与算法描述完全一致，差异仅体现在任务划分与线程组织方式上，因此其性能差异可直接归因于并行结构本身。各版本说明如下：

- v1（单线程顺序实现，llh_hash_v1）：基准版本，在单一线程内顺序完成全部吸收与挤压运算，不涉及任何并发，用作衡量各并行版本加速比的统一参照。
- v2（多线程并行归并实现，ParallelHash）：借鉴 ParallelHash 的树形结构，将输入消息切分为若干分块后交由多个线程并行计算各分块摘要，再将中间结果归并为最终杂凑值。
- v3（多线程块状并行归并实现，K12-style）：仿照 KangarooTwelve（K12）的块状并行结构组织计算，在分块并行的基础上为短消息提供专门的处理路径，以降低小规模输入下的并行开销。

注：多线程版本的线程数在本地桌面平台固定为 4，在树莓派平台则由系统可用核数决定。

6.1.2.1 测试环境及用例

为保证测试结果的可复核性，本节在本地桌面平台与树莓派平台上分别独立运行相同的测试脚本，并在每个平台内部统一固定操作系统、CPU、编译器版本、编译优化选项以及测试框架，从而保证同一平台内不同实现版本之间的可比性。两个平台的硬件与软件配置分别如表 6-9 与表 6-10 所示。

表 6-9 本地平台测试环境

项目	规格
操作系统	Linux Ubuntu 22.04
CPU	Intel(R) Core(TM) i7-12700
编译器	GCC (-O2 优化)
测试框架	Python 3 + subprocess

表 6-10 树莓派平台测试环境

项目	规格
操作系统	Raspberry Pi OS (Linux ARM)
CPU	ARM Cortex-A 系列 (低主频多核)
编译器	GCC (-O2 优化)
测试框架	Python 3 + subprocess

测试用例分为两组，定义见表 6-11。第一组用于刻画短消息区间内性能随输入长度的细粒度变化，覆盖 1B 至 512B（其中 512B 等于 4096 比特，即 2 的 12 次方比特）的全部输入长度，步长为 1B；第二组 (Test2) 用于反映典型消息规模下的总体性能，分别取 1KB、1MB 与 1GB 三个数据规模。上述测试均在 LLH-256、LLH-512、LLH-768 与 LLH-1024 四个标准实例上分别执行，每个数据规模独立重复测量 100 次并取算术平均值作为最终结果，从而覆盖不同字宽与不同并行粒度下的性能表现。

表 6-11 树莓派平台测试环境

编号	数据规模
Test1	1 - 512B, 步长 1B
Test2	1KB 、 1 MB、 1 GB

6.1.2.2 测试流程

本地平台与树莓派平台采用相同的测试流程，具体步骤如下：

- 生成指定规模的随机输入数据；
- 依次运行 v1 单线程版本与 v2、v3 多线程版本，并在线程池完成预热后开始计时；
- 记录每次运行的完整耗时，包括数据读取时间（`read_time`）与计算时间（`compute_time`）；
- 对每个数据规模独立重复测量 100 次，取算术平均值作为该组最终结果。

为保证测试结果的有效性与严谨性，测试过程中严格控制变量：其一，同一组对比始终保持相同平台、相同编译器版本与相同测试框架，确保架构与工具链一致；其二，多线程版本的线程数固定，不允许运行期动态调度，以避免引入额外的随机性误差；其三，本地平台与树莓派平台分别独立运行相同的测试脚本，保证两类环境彼此隔离、互不干扰。

6.1.2.3 测试结果

本节基于两类硬件平台，分别从吞吐量、加速比以及按实例细分的加速比三个角度给出测试结果。相关性能指标的定义如下：

- 执行时间：算法完整执行所花费的时长，等于数据读取时间与计算时间之和（`read_time + compute_time`，单位为 ms）；
- 吞吐率：单位时间内处理的数据量，等于数据量除以执行时间（单

位为 MB/s)；

- 加速比：v1 基准版本的执行时间与并行版本执行时间之比，数值大于 1 表示并行实现相对基准取得了加速效果。
- 基于本地平台的测试结果

图 6-12 给出 Test1（即 1B~512B 短消息区间）下，各标准实例采用 ParallelHash 与 K12-style 时相对 v1 基准的细粒度加速比曲线。可以看到，ParallelHash 的加速比始终小于 1：在如此小的数据规模下，线程创建与管理引入的固定开销远大于并行计算所节省的时间，因此并行反而拖慢了整体执行。K12-style 的加速比虽存在一定波动，但整体大于 1，说明其面向短消息优化的执行路径对加密速度具有正向作用。K12-style 曲线不够平稳的原因可从软硬件两方面理解：从算法结构看，在极小输入规模下，部分实例的分块组织会引入额外的调度、分支与数据整理成本，难以触发更高效的 SIMD 向量化路径，从而出现局部性能劣势；从硬件层面看，本地平台采用 Alder Lake 异构架构（P-core 与 E-core 并存），进一步放大了这种波动——小任务的执行时间极短，线程调度、核心迁移以及一级、二级缓存的命中状态都会显著影响总耗时，一旦偶发尾块处理、分支预测失误或被调度到能效核（E-core）上执行，整体性能甚至可能低于基准实现。

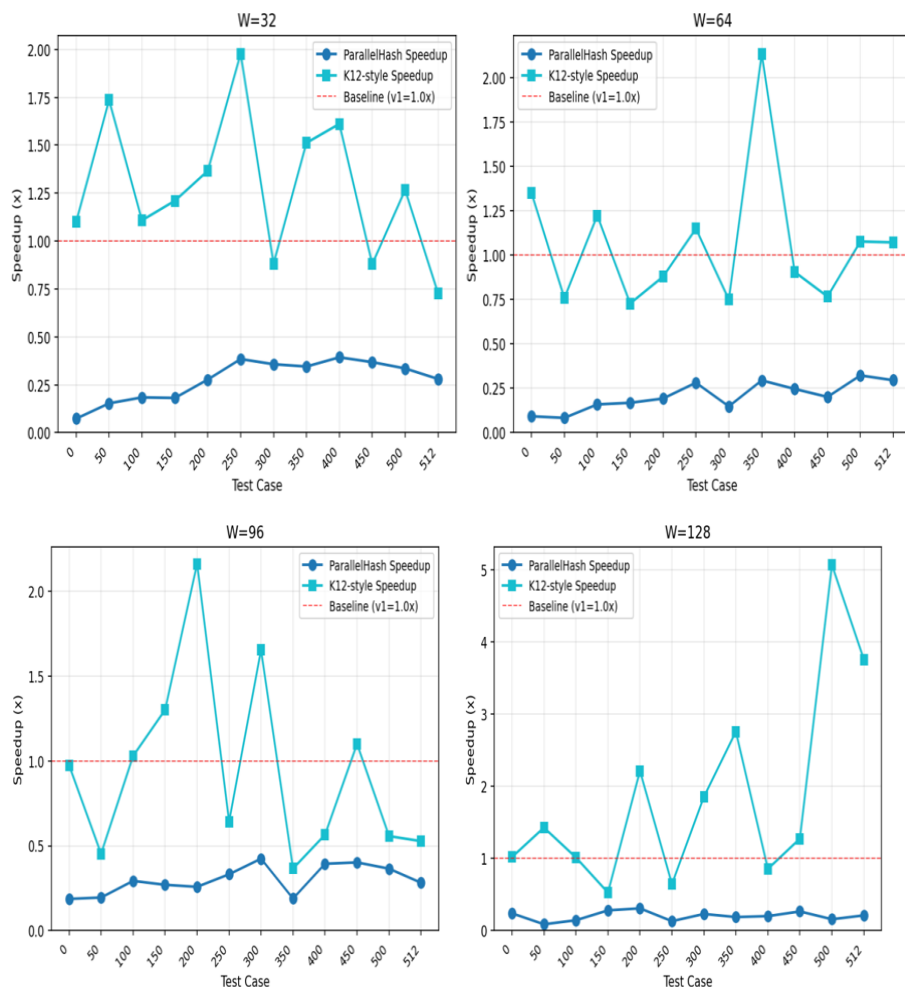


图 6-12 本地平台：Test1 (1B~512B) 下各标准实例的细粒度加速比曲线 (ParallelHash 与 K12-style)

图 6-13 给出 Test2 中 1KB、1MB 与 1GB 三个数据规模下，各标准实例三种实现版本的吞吐量对比。当数据规模为 1KB 和 1MB 时，ParallelHash 的吞吐量低于 v1 基准，其原因依旧是小规模数据下线程创建开销远高于实际哈希计算时间；当数据规模增大到 1GB 时，其吞吐量提升明显，加速比接近所用线程数量。相比之下，K12-style 在数据规模达到 1MB 时即已表现出明显的加速效果。这是因为随着数据规模增大，并行调度、线程创建、缓存冷启动等固定开销被逐渐摊薄（amortization effect）；这些在小数据区间占主导地位的开销，在 GB 级输入下几乎可

以忽略，从而使 K12-style 的结构优势得以充分释放。

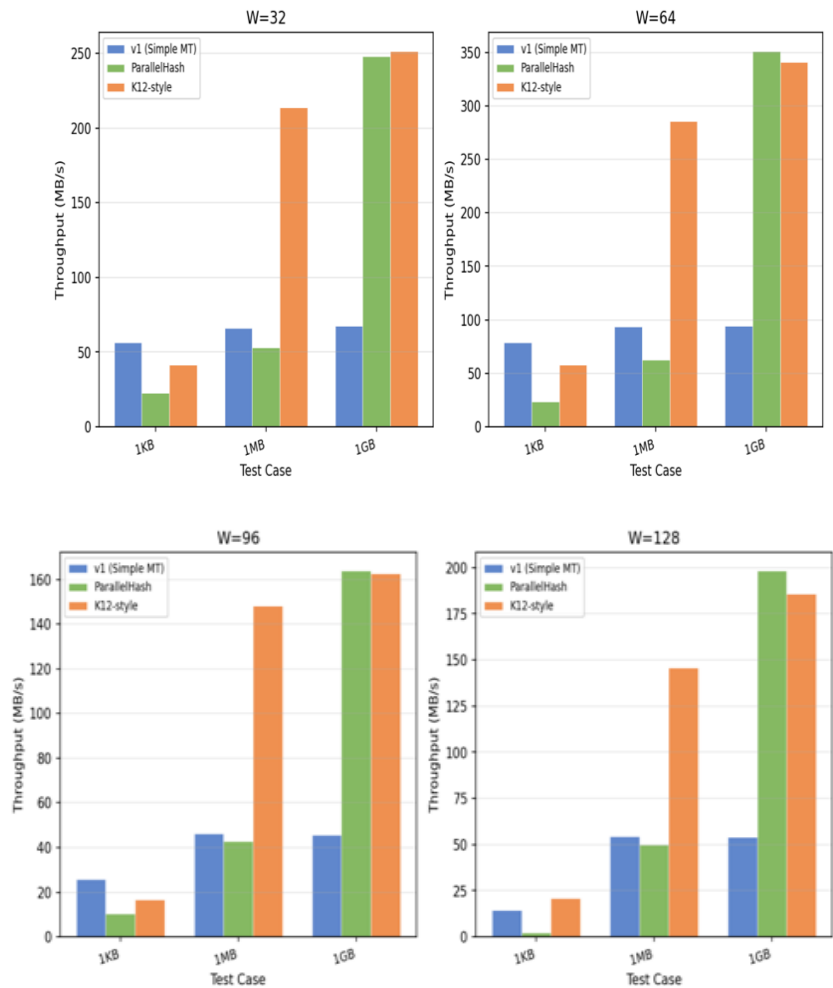


图 6-13 本地平台：1KB、1MB、1GB 下三种实现版本的吞吐量对比
(各标准实例，单位 MB/s)

图 6-14 进一步以加速比为指标，给出本地平台在 1KB、1MB 与 1GB 三个数据规模下 ParallelHash 与 K12-style 相对 v1 基准的加速比，可与图 6-13 的吞吐量结果相互印证。结合表 6-12 至表 6-14 的实测数据可以看出，二者的加速比随数据规模呈现出清晰的演化规律。在 1KB 规模下，ParallelHash 的加速比在各实例上均低于 1，K12-style 也大多低于 1，表明此时线程创建与调度的固定开销仍高于并行所节省的计算时

间；当数据规模增大至 1MB 时，ParallelHash 受归并与线程管理开销影响，加速比仍未超过 1，而 K12-style 凭借面向块状并行的精简结构率先取得明显加速；当数据规模进一步增大至 1GB 时，二者的固定开销被充分摊薄，加速比共同收敛至约 3.4~3.7×，接近本地平台所用线程数在存在调度开销下的实际可达上限。由于本地平台具有充足的内存带宽与多级缓存，系统在大数据规模下仍主要表现为计算受限，因此并行结构的性能优势能够随数据规模稳定释放，这与图 6-13 所反映的吞吐量变化趋势完全一致。

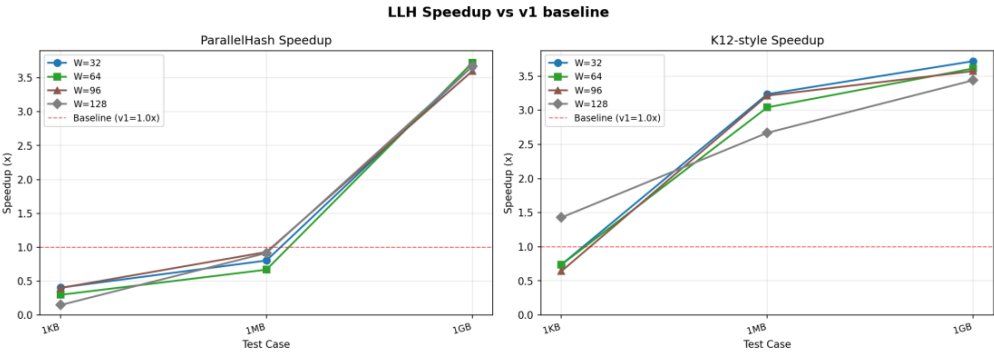


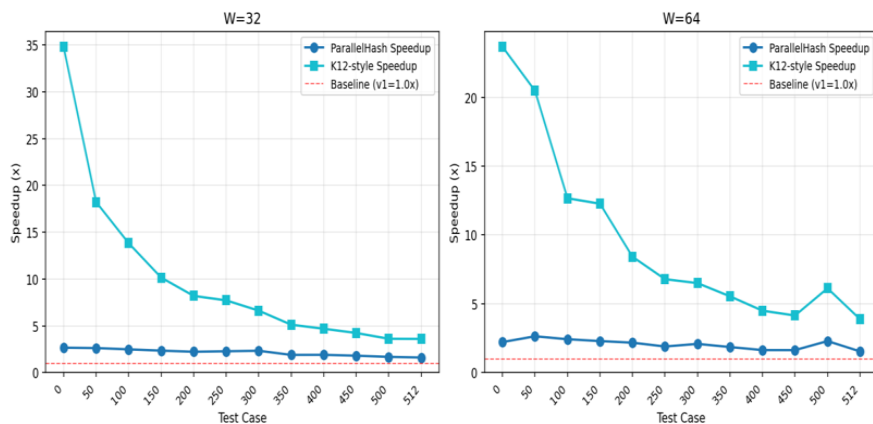
图 6-14 本地平台：1KB、1MB、1GB 下 ParallelHash 与 K12-style 相对 v1 基准的加速比

● 基于树莓派平台的测试结果

以下三图给出树莓派 ARM 平台上的相同指标，用于与本地平台结果形成对照。与配备充足内存带宽的本地桌面平台不同，树莓派采用低主频、多核的 Cortex-A 系列处理器，其内存带宽与缓存容量相对受限，因而在不同数据规模下的并行行为与本地平台存在明显差异，可据此考察并行化策略在资源受限嵌入式环境中的实际可扩展性。

图 6-15 给出树莓派平台上 Test1（1B~512B 短消息区间）下各标准实例的细粒度加速比曲线。针对该小数据区间的测试表明，ParallelHash

与 K12-style 均能稳定取得性能提升,但二者的加速特性存在本质差异: ParallelHash 的加速比平均约为 2,表现出对输入规模不敏感的稳定增益; K12-style 在极小输入时加速比可高达约 $35\times$,但随着数据规模增加逐步回落至约 $2\times$ 。这一现象说明,在资源受限且并行能力有限的硬件环境中,小数据性能主要由实现路径优化而非并行结构本身决定: ParallelHash 的通用树形结构在小输入下难以发挥并行优势,因而仅体现为轻量级的实现优化收益;而 K12-style 凭借面向短消息的专门优化路径在极小规模下获得高效执行,但其优势会随计算负载的增加被逐渐摊薄。总体而言,这种差异反映了两类设计取向的不同——ParallelHash 偏重结构的通用性与可扩展性, K12-style 则更强调跨规模、尤其是短消息场景下的实际性能表现。



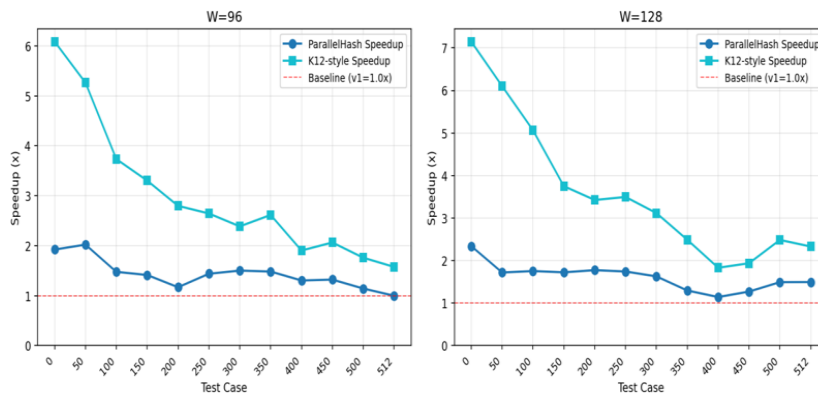


图 6-15 树莓派平台：Test1 (1B~512B) 下各标准实例的细粒度加速比曲线 (ParallelHash 与 K12-style)

图 6-16 给出树莓派平台上 1KB、1MB 与 1GB 三个数据规模下各标准实例三种实现版本的吞吐量对比。当数据规模增大至 1MB 和 1GB 时，无论 ParallelHash 还是 K12-style，其加速比均逐渐趋近于 1，表明在大规模输入下并行结构带来的性能优势基本消失。这一现象说明系统已从“计算受限” (compute-bound) 转变为“内存带宽受限” (memory-bound)：在小数据区间，性能主要由算法路径、函数调用开销与指令级优化决定，不同实现之间可以体现出明显差异；而当输入规模扩大到 MB 乃至 GB 级别时，内部轮函数的计算成本被大量数据访问所掩盖，系统瓶颈转移至内存子系统，此时无论是串行结构还是具备树形并行能力的版本，都需要持续从内存读取数据并写回状态，整体吞吐受限于内存带宽与缓存层级结构，而非算法本身。

在树莓派的 Cortex-A 系列架构上，这一现象尤为明显：其内存带宽与缓存容量相对有限，多核并行在处理大规模流式数据时会产生带宽竞争 (bandwidth contention)，导致各线程无法同时以满速运行。结果是并行化所能带来的理论加速被内存访问瓶颈完全抵消，不同实现的吞吐率趋于收敛，加速比逼近 1。这一结果与本地服务器平台的测试结果形

成了明确对照。

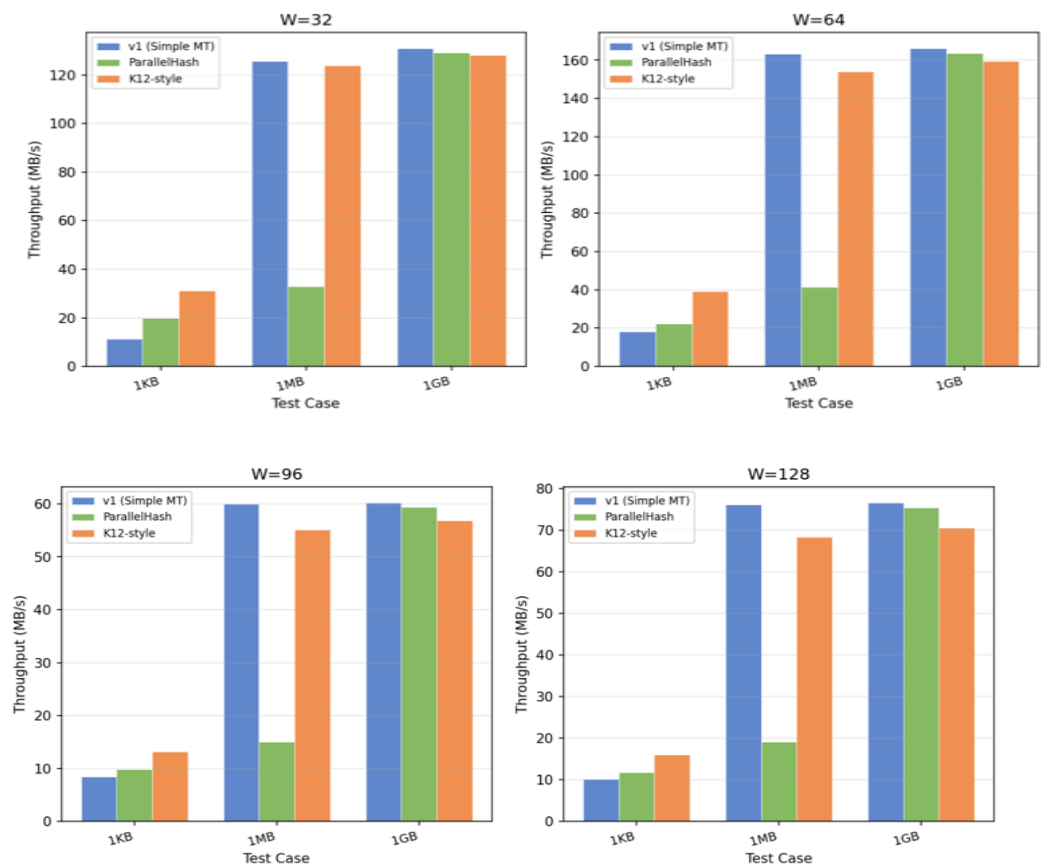


图 6-16 树莓派平台：1KB、1MB、1GB 下三种实现版本的吞吐量对比
(各标准实例，单位 MB/s)

图 6-17 以加速比为指标，给出树莓派平台在 1KB、1MB 与 1GB 三个数据规模下 ParallelHash 与 K12-style 相对 v1 基准的加速比，可与图 6-16 的吞吐量结果相互印证。由表 6-15 至表 6-17 的实测数据可见，树莓派平台的加速比随数据规模呈现出与本地平台相反的非单调规律。在 1KB 小数据规模下，ParallelHash 与 K12-style 的加速比均高于 1（ParallelHash 约 1.15~1.76，K12-style 约 1.55~2.75），说明在资源受限的 ARM 架构上，短消息区间的性能主要由实现路径优化决定，二者相对串行基准均能取得正向收益；当数据规模增大至 1MB 时，二者的加

速比反而同时跌至 1 以下，其中 ParallelHash 骤降至约 0.25，明显劣于 v1 基准，K12-style 也回落至约 0.90~0.99，表明此时多核并行非但未能提速，反而因线程归并与数据搬运开销引入了额外代价；当数据规模进一步增大至 1GB 时，加速比有所回升，ParallelHash 恢复至约 0.985，K12-style 约 0.92~0.98，但二者始终未能超过基准。上述现象与图 6-16 的吞吐量结果一致：树莓派采用的 Cortex-A 系列架构内存带宽与缓存容量相对有限，当输入规模扩大到 MB 乃至 GB 级别时，系统由计算受限转为内存带宽受限，多核并行在处理大规模流式数据时产生带宽竞争，使并行化的理论加速被内存访问瓶颈抵消，因而无论 ParallelHash 还是 K12-style，其加速比都只能逼近而无法稳定超过 1。这一跨平台对照清楚地表明，并行化策略的实际收益高度依赖底层硬件的内存子系统特性。

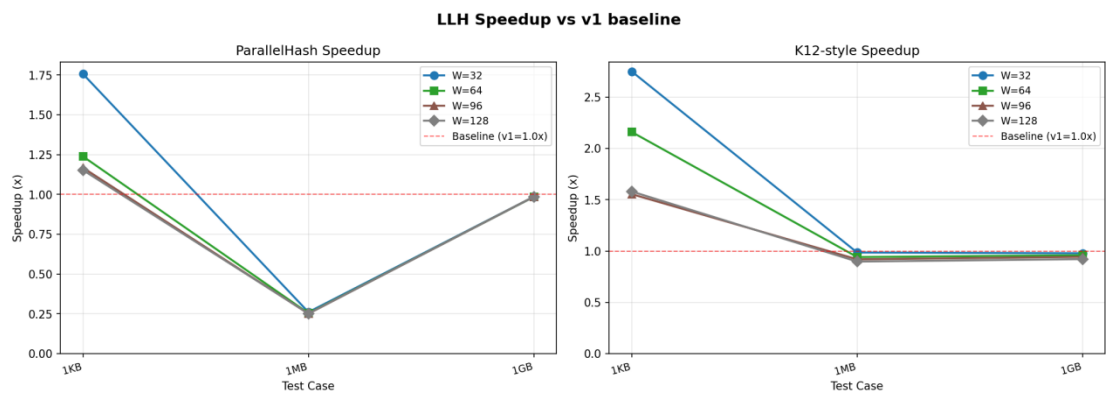


图 6-17 树莓派平台：1KB、1MB、1GB 下 ParallelHash 与 K12-style 相对 v1 基准的加速比

6.1.2.4 原始数据摘要

本节列出上述测试的原始实测数据，所有数值均为每组重复 100 次测量后的均值；执行时间单位为 ms，加速比为无量纲数值，大于 1 表示相对 v1 单线程基准取得加速效果。为与全文实例命名保持一致，下列各表以 LLH-256、LLH-512、LLH-768、LLH-1024 标识字宽分别为 w=32、

64、96、128 的标准实例。

• 三版本对比实测数据——本地平台

下列各表给出三种实现版本在本地平台上的执行时间及其各自加速比（以 v1 加速比=1.00 为基准，PH 与 K12 加速比大于 1 表示优于 v1）。

表 6-12 数据规模：1KB

实例	v1 (ms)	v1 加速比	PH (ms)	PH 加速比	K12 (ms)	K12 加速比
LLH-256	0.017	1.00	0.043	0.404	0.023	0.739
LLH-512	0.012	1.00	0.042	0.298	0.017	0.738
LLH-768	0.038	1.00	0.095	0.399	0.059	0.642
LLH-1024	0.067	1.00	0.455	0.147	0.047	1.429

表 6-13 数据规模：1MB

实例	v1 (ms)	v1 加速比	PH (ms)	PH 加速比	K12 (ms)	K12 加速比
LLH-256	15.124	1.00	18.812	0.804	4.672	3.237
LLH-512	10.639	1.00	15.906	0.669	3.496	3.043
LLH-768	21.674	1.00	23.309	0.930	6.741	3.215
LLH-1024	18.319	1.00	20.030	0.915	6.862	2.670

表 6-14 数据规模：1GB

实例	v1 (ms)	v1 加速比	PH (ms)	PH 加速比	K12 (ms)	K12 加速比
LLH-256	15147.932	1.00	4125.093	3.672	4072.121	3.720
LLH-512	10859.831	1.00	2916.643	3.723	3006.068	3.613
LLH-768	22482.521	1.00	6245.006	3.600	6290.129	3.574
LLH-1024	18938.811	1.00	5160.865	3.670	5505.842	3.440

• 三版本对比实测数据——树莓派平台

下列各表给出树莓派平台上的相同指标，与本地平台结果形成跨平台对照。

表 6-15 数据规模：1KB

实例	v1 (ms)	v1 加速比	PH (ms)	PH 加速比	K12 (ms)	加速比
LLH-256	0.086	1.00	0.049	1.757	0.031	2.751
LLH-512	0.054	1.00	0.043	1.239	0.025	2.161
LLH-768	0.115	1.00	0.099	1.168	0.074	1.555
LLH-1024	0.096	1.00	0.083	1.153	0.061	1.581

表 6-16 数据规模：1MB

实例	v1 (ms)	v1 加速比	PH (ms)	PH 加速比	K12 (ms)	加速比
LLH-256	7.959	1.00	30.453	0.261	8.073	0.986
LLH-512	6.116	1.00	24.038	0.254	6.495	0.942
LLH-768	16.683	1.00	66.369	0.251	18.139	0.920
LLH-1024	13.145	1.00	52.217	0.252	14.637	0.898

表 6-17 数据规模：1GB

实例	v1 (ms)	v1 加速比	PH (ms)	PH 加速比	K12 (ms)	加速比
LLH-256	7817.919	1.00	7928.892	0.986	7986.995	0.979
LLH-512	6160.203	1.00	6252.141	0.985	6420.032	0.960
LLH-768	17001.448	1.00	17247.321	0.986	18013.110	0.944
LLH-1024	13371.120	1.00	13568.132	0.985	14500.361	0.922

6.1.2.5 结果分析

综合两类平台的实测结果，可得出以下分析结论。

- 大数据场景下并行加速效果显著（本地平台）：在 1GB 数据规模下，ParallelHash 与 K12-style 相对 v1 的加速比均稳定处于约 $3.4 \times \sim 3.7 \times$ 之间，且在四个标准实例上表现一致，说明在 CPU 密集型的大数据哈希任务中，多线程并行能够带来明显且可预期的性能收益。
- 小数据场景下并行开销抵消收益（本地平台）：在 1KB 数据规模下，并行版本的加速比普遍低于 1，线程创建与调度的固定开销

超过了并行计算所节省的时间，多线程版本反而慢于单线程基准。

- 树莓派平台呈现相反规律：在 1KB 小数据场景下，K12-style 仍可获得约 $1.5\times\sim 2.7\times$ 的加速；但在 1MB 和 1GB 场景下，各并行版本的加速比接近 1 甚至略低于基准，说明此时线程调度开销已与计算量相当，内存带宽成为制约性能的主要因素，其中 ParallelHash 在大数据下甚至略慢于 v1。综合来看，并行化收益高度依赖平台的算力与访存特性，需结合具体硬件环境与消息规模选择合适的实现版本。

6.2 硬件性能评估

6.2.1 硬件实现

LLH 算法硬件模块由 AXI 总线接口、数据填充以及吸收与挤压三个主要单元构成。其中核心模块为吸收与挤压单元，是影响算法性能的关键模块，也是算法评估的核心，其它两个单元在实现上还存在大量的优化空间，在评估中并不刻意关注。

需要特别说明的是，本算法支持四种不同的长度格式，在实现中通过软件配置实现，导致较短格式下的功耗和实现开销比实现单一长度格式的电路更高。在实际应用中，可以通过针对特定长度进行实现以满足相关需求。

AXI 总线接口单元支持 128 bit 数据读写，当数据位宽超过 128 bit 时需发起多次总线操作。

数据填充单元依据算法规定的填充规则设计：每当累计输入数据达到比特率值时，即输出一组消息至吸收与挤压单元；若最终一组输入数据长度不足比特率值，则对其进行填充处理。

吸收与挤压单元作为核心算法运算部件，为有效缩减实现面积并提升运行效率，其轮函数采用组合逻辑方式实现，并通过有限状态机控制各轮的数据传递及吸收与挤压状态的转移。

根据算法设计原理，吸收与挤压模块在获得消息长度后无须等待数据输入，即可开始计算 $P(S^{(0)})$ ，从而能够在算法层面掩盖第一组数据的输入开销，本设计最大程度节省了输入数据所消耗的时间，确保整体运算不需要多余的时钟周期。

6.2.2 运算效率

算法	时钟周期数 (cycles)							
	32 B	128 B	512 B	1 KB	4 KB	8 KB	16 KB	64 KB
LLH-256	33	81	273	529	2065	4113	8209	32785
LLH-512	49	73	217	409	1561	3097	6169	24601
LLH-768	65	97	225	385	1409	2785	5505	21889
LLH-1024	81	81	201	361	1321	2601	5161	20521

算法	归一化性能 (bit/cycle)							
	32 B	128 B	512 B	1 KB	4 KB	8 KB	16 KB	64 KB
LLH-256	7.7576	12.6420	15.0037	15.4858	15.8683	15.9339	15.9669	15.9917
LLH-512	5.2245	14.0274	18.8756	20.0293	20.9917	21.1611	21.2469	21.3117
LLH-768	3.9385	10.5567	18.2044	21.2779	23.2562	23.5318	23.8096	23.9521
LLH-1024	3.1605	12.6420	20.3781	22.6925	24.8055	25.1965	25.3966	25.5489

当数据量达到 1024Bytes 后，性能已经较高，逐渐接近算法理论峰值。

6.2.3 硬件实现面积(12nm)

综合后总面积：

模块	面积 (μm^2)
----	------------------------

配置模块	293.1794
填充模块	5960.3928
吸收与挤压模块	13768.1513
总计	20039.307572

综合后模块面积：

模块	总面积(μm^2)	百分比	组合逻辑面积(μm^2)	非组合逻辑面积(μm^2)
顶层模块	20039.3076	100	17.5841	0
配置模块	293.1794	1.5	189.6284	100.8968
填充模块	5960.3928	29.7	4977.5617	979.2922
吸收与挤压模块	13768.1513	68.7	8309.7724	4349.5835

工艺库中最小尺寸 NAND2 逻辑门面积为 $0.11059\mu\text{m}^2$ ，选其作为归一化计算参数。计算得到以下归一化面积：

模块	NAND2 等效门数
配置模块	2651.0480
填充模块	53896.3089
吸收与挤压模块	124497.2538
总计	181203.6131

6.2.4 硬件时延(12nm)

算法电路的关键路径为内部状态经过置换函数后得到新一轮的内部状态，根据综合工具实现的结果，该路径的最大延迟为 0.64ns ，即满足 STA 收敛的最高工作频率为 1.56Ghz 。

6.2.5 吞吐量

算法	运算吞吐量 (Mb/s)							
	32 B	128 B	512 B	1 KB	4 KB	8 KB	16 KB	64 KB

LLH-256	12121.21	19753.08	23443.22	24196.60	24794.19	24896.67	24948.23	24987.04
LLH-512	8163.27	21917.81	29493.09	31295.84	32799.49	33064.26	33198.25	33299.46
LLH-768	6153.85	16494.85	28444.44	33246.75	36337.83	36768.40	37202.54	37425.19
LLH-1024	4938.27	19753.09	31840.80	35457.06	38758.52	39369.47	39682.23	39920.08

上述吞吐量为 1.56Ghz 时钟频率下,不同输入数据长度的仿真结果。根据仿真结果,随着输入数据量的增加,模块的整体吞吐量仍呈现上升趋势,当数据量超过 8KB 后,吞吐量增长趋势变缓,接近算法理论峰值。

6.2.6 归一化吞面比

工艺库中最小尺寸 NAND2 逻辑门面积为 $0.11059 \mu m^2$, 选其作为归一化计算参数。计算公式为:

归一化吞面比=运算吞吐量/(综合后总面积/NAND2 逻辑门面积)

下表为不同数据量下的归一化吞面比:

算法	归一化吞面比(Mb/s/NAND2 等效门数)							
	32 B	128 B	512 B	1 KB	4 KB	8 KB	16 KB	64 KB
LLH-256	0.0669	0.1090	0.1294	0.1335	0.1368	0.1374	0.1377	0.1379
LLH-512	0.0451	0.1210	0.1628	0.1727	0.1810	0.1825	0.1832	0.1838
LLH-768	0.0340	0.0910	0.1570	0.1835	0.2005	0.2029	0.2053	0.2065
LLH-1024	0.0273	0.1090	0.1757	0.1957	0.2139	0.2173	0.2190	0.2203

6.2.7 功耗

零工作负载时的功耗为:

模块	LLH-256/512/768/1024
配置模块	47.03uW
填充模块	72.48uW

吸收与挤压模块	1.73mW
总计	1.85mW

满工作负载时的功耗为：

	LLH-256	LLH-512	LLH-768	LLH-1024
配置模块	80.77 uW	95.59 uW	99.94 uW	779.82 uW
填充模块	848.29 uW	1.57 mW	2.67 mW	2.57 mW
吸收与挤压模块	54.32 mW	59.22 mW	63.88 mW	67.16 mW
总计	55.25 mW	60.88 mW	66.65 mW	70.51 mW

总功耗为：

	LLH-256	LLH-512	LLH-768	LLH-1024
配置模块	127.8 uW	142.62 uW	146.97 uW	826.85 uW
填充模块	920.77 uW	1.64 mW	2.74 mW	2.64 mW
吸收与挤压模块	56.05 mW	60.95 mW	65.61 mW	68.89 mW
总计	57.1 mW	62.73 mW	68.5 mW	72.36 mW

算法	归一化功耗（Mb/s/mW）							
	32 B	128 B	512 B	1 KB	4 KB	8 KB	16 KB	64 KB
LLH-256	88.22	143.77	170.62	176.11	180.46	181.20	181.58	181.86
LLH-512	54.08	145.20	195.39	207.33	217.30	219.05	219.94	220.61
LLH-768	37.33	100.07	172.57	201.71	220.46	223.07	225.71	227.06
LLH-1024	28.36	113.45	182.87	203.64	222.60	226.11	227.91	229.27

6.2.8 性能分析

在算法性能层面，当前实现的 AXI 接口为 128 bit 宽度，总线利用率低。可通过支持突发长度的 AXI Burst 传输，并增加写入与读出

FIFO，实现在一次地址握手后连续搬运多拍数据，从而有效提升总线利用率，降低接口连接开销。

从实现性能层面，本算法轮函数的设计体现了超低时延的目标，从上述综合结果也可以看到，其轮函数关键路径长度较低，在设计频率不需要太高的情况下，可以考虑将轮函数进行多轮展开，以降低实际运行的总周期数。此类优化的主要代价在于展开逻辑带来的面积增长及设计复杂度，但频率由于实现优化通常不会线性下降，而带来性能上的好处。

从算法配置模式层面，本算法支持四种不同长度模式，在实际应用中，可以通过针对特定长度进行实现以达到资源、频率、效率和功耗的最优化。

7 算法创新性和优势

7.1 算法创新性

LLH 算法是一族基于海绵结构变体设计的超低时延密码杂凑算法，其创新主要体现在：

突破了大尺寸、高扩散、低时延非线性置换构造理论：我们推广了一般循环电路函数，给出了大尺寸、高扩散度、最低时延的非线性置换构造方法(参考定理 2-1)，从而为设计超低时延密码算法奠定了理论基础。

超低时延杂凑算法：LLH 算法具有超低的电路实现时延，其主要得益于结构选型和基础组件设计。在结构选型方面，我们选择基于切片技术构造 P 置换的轮函数。该构造方式在行方向的线性扩展和常数加的电路实现时延可以忽略不计，其时延主要来自于切片方向的变换。在切片

方向，我们进一步通过 SPN 结构设计将其分解为小 S 盒变换、错位排列和行线性变换等三个小组件操作，其中 6 比特小 S 盒的时延做到了非线性置换数学理论的下界，错位排列在硬件电路实现上没有时延，行线性变换在分支数达到最佳时时延同样达到了数学理论下界。从而，整个 P 置换轮函数的电路实现深度仅为 4，在扩散分支数为 5 时几乎达到了最优。上述构造使得 LLH 算法的电路实现时延非常低，是目前已知杂凑算法中时延最低的。

前馈海绵结构变体：LLH 算法采用前馈海绵结构变体，将消息长度编码到初始状态中，不仅继承了原海绵结构的安全性优点，还进一步提升了 LLH 算法抗与消息扩展有关攻击的能力。

7.2 算法优势

与已知杂凑算法标准 SM2、SHA2、SHA-3 相比，LLH 算法具有如下优势：

- LLH 算法具有非常低的硬件实现时延，是目前已知杂凑算法中时延最低的。
- LLH 算法采用前馈海绵结构变体设计，将消息长度编码到初始状态中，不仅继承了海绵结构的优点，具有较高的安全特性，还进一步提升了 LLH 算法抗消息扩展有关攻击的能力。
- LLH 算法主要采用软硬件实现友好的异或、逻辑与和循环移位等基础运算，具有非常低的软硬件实现成本和非常高的软硬件效率。此外，整个算法低深度的构造方式非常有利于 SIMD 等向量指令加速，多种优化方式可满足不同平台的实现需求。
- LLH 算法具有非常高的抗侧信道攻击的能力。在硬件设计侧未引入

任何侧信道防护机制的条件下，面对当前最强侧信道攻击，LLH 算法能达到理论上的最小泄漏量。

综上，LLH 算法主要追求极致的低时延，在安全、实现、软硬件实现成本和效率等方面取得了较好的平衡。这种平衡性正是 LLH 算法作为超低时延密码杂凑算法候选方案的核心价值所在。

8 结论

LLH 是一族具有超低时延的密码杂凑算法。该算法采用前馈海绵结构变体，支持 256、512、768 和 1024 比特四种不同长度的输出杂凑值。其整体设计围绕“低时延、易实现、可分析”三项目标展开：在结构层面采用统一而规整的切片迭代结构，在组件层面采用达到理论下界的基础非线性与线性部件，在轮函数层面通过列替换、行移位和常数加实现非线性与扩散的协同传播。

从设计结果看，LLH 算法的最突出特点是其低时延优势具有明确的理论依据。基础 6 比特 S 盒和 4 比特线性变换都达到了相应数学理论下界，由此构成的 24 比特超级 S 盒和轮函数保持了可追踪的低深度结构。因此，LLH 算法的“超低时延”并不是经验层面的实现优化，而是自底向上层层支撑的理论设计结果。结合第 6 章的软硬件性能评估，可以认为 LLH 算法在面向低时延应用场景时具有明显的结构优势。

从安全性分析结果看，LLH 已经形成了较为完整的分层评估框架。基础组件、超级 S 盒、轮函数和整体模式的主要性质都得到了系统分析；在经典安全性方面，给出了低轮迭代的安全边界；在抗量子计算方面，模式级单向性已经可以归约到量子随机置换模型下的公开切片单轮迭代，而实例级缩减轮分析目前尚未给出能够推翻标准参数后量子安全

目标的全轮结构化攻击。因此，我们断言，LLH 算法安全边界在当前研究范围内总体上是清晰且可解释的。

从应用角度看，LLH 的规整结构和软硬件实现友好性，使其既适合软件 SIMD 环境，也适合对组合逻辑深度和门级时延敏感的硬件平台。对资源受限设备、超低时延认证场景以及对杂凑运算时延较为敏感的系统而言，LLH 具有较好的应用潜力。

综上所述，LLH 在设计理念、密码组件选择、整体结构构造、安全性分析和实现性能之间形成了较好的统一。其主要创新在于构造了一条以“可证明的最低时延”为核心目标的设计路径，并在此基础上兼顾了扩散能力、结构透明性和安全可分析性。基于目前的理论分析与实验评估结果，可以认为 LLH 是一类具有明确特色、具备进一步研究和应用评估价值的超低时延密码杂凑算法方案。

致谢

谨向刘俊锐博士，博士研究生徐锐、马若为、陈新、张嘉宁，硕士研究生苏家齐、罗澜、佟京铖、王传刚、姜佳怡表示感谢，感谢他们参与 LLH 算法的设计、实现与性能评估工作，并在算法讨论、实验验证及报告完善过程中提供了宝贵的意见和支持。他们的贡献对本报告的完成起到了重要作用，在此致以诚挚的谢意。

参考文献

- [1] Bertoni G, Daemen J, Peeters M, et al. Sponge functions[C]//ECRYPT hash workshop. 2007, 2007(9).
- [2] Chen S, Xiang Z, Zeng X, Zhang S: On the Relation-ships between Different Methods for Degree Evaluation. IACR Transactions on Symmetric Cryptology, 2021(1), 411-442. <https://doi.org/10.46586/tosc.v2021.i1.411-442>
- [3] Bertoni G, Daemen J, Peeters M, et al. On the indifferentiability of the sponge construction[C]//Annual International Conference on the Theory and Applications of Cryptographic Techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008: 181-197.
- [4] Damgård I B. A design principle for hash functions[C]//Conference on the Theory and Application of Cryptology. New York, NY: Springer New York, 1989: 416-427.
- [5] Rogaway P, Shrimpton T. Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance[C]//International workshop on fast software encryption. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004: 371-388.
- [6] Mouha N. Automated techniques for hash function and block cipher cryptanalysis[J]. Belgium: Khatolieke Universiteit Leuven, 2012.
- [7] Mironov I, Zhang L. Applications of SAT solvers to cryptanalysis of hash functions[C]//International Conference on Theory and Applications of Satisfiability Testing. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006: 102-115.
- [8] Kocher P, Jaffe J, Jun B. Differential power analysis[C]//Annual

- international cryptology conference. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999: 388-397.
- [9] Brier E, Clavier C, Olivier F. Correlation power analysis with a leakage model[C]//International workshop on cryptographic hardware and embedded systems. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004: 16-29.
 - [10] Bogdanov A, Kizhvatov I, Manzoor K, et al. Fast and memory-efficient key recovery in side-channel attacks[C]//International Conference on Selected Areas in Cryptography. Cham: Springer International Publishing, 2015: 310-327.
 - [11] Grover L K. A fast quantum mechanical algorithm for database search[C]//Proceedings of the twenty-eighth annual ACM symposium on Theory of computing. 1996: 212-219.
 - [12] Bennett C H, Bernstein E, Brassard G, et al. Strengths and weaknesses of quantum computing[J]. SIAM journal on Computing, 1997, 26(5): 1510-1523.
 - [13] Amy M, Di Matteo O, Gheorghiu V, et al. Estimating the cost of generic quantum pre-image attacks on SHA-2 and SHA-3[C]//International Conference on Selected Areas in Cryptography. Cham: Springer International Publishing, 2016: 317-337.
 - [14] Feng X, Wang Q, Zhang A, Jiang Y: On the construction of ultra-low latency nonlinear permutation. preprint.